System Design Document

ZeoAuction

Distributed Real-Time Auction Platform

Microservices / Service-Oriented Architecture

Document type	System Design Document (SDD) — High- and Low-Level Design
Version	1.0
Date	August 2026
Related document	ZeoAuction Software Requirements Specification v1.0

Author	**Design area**
Naitik (202512006)	Bid Processing; Integration Testing
Sumit (202512124)	Auction Orchestration; Documentation
Sagar (202512107)	KYC & Seller Verification; Pricing Engine; Search & Analytics
Harshal (202512013)	Fraud Detection
Jaydip (202512090)	Payment, Fees & Escrow
Gaurang (202512007)	Frontend
Jay (202512123)	Dispute Management; Admin & Moderation
Ayush (202512084)	Auth & User Service; WebSocket Gateway; Infrastructure

This document specifies the architecture, data design, component and class design, behavioural design, REST API contracts and final UI/UX mockups that realise the requirements of the ZeoAuction SRS.

Contents

1 Introduction **8**

1.1 Purpose . . . 8

1.2 Scope . . . 8

1.3 Design Baseline . . . 9

1.4 Definitions Used in This Document . . . 9

1.5 Document Conventions . . . 10

2 Architectural Overview **11**

2.1 Design Drivers . . . 11

2.2 System Context . . . 11

2.3 Container Architecture . . . 12

2.3.1 Tiers . . . 13

2.3.2 Why These Service Boundaries . . . 14

2.4 Synchronous and Asynchronous Interaction . . . 14

2.5 Event and Topic Topology . . . 15

2.6 Architecture Decision Record Summary . . . 16

3 Deployment Architecture **19**

3.1 Deployment Topology . . . 19

3.1.1 Render . . . 19

3.1.2 Vercel . . . 20

3.1.3 The Load-Test Host . . . 20

3.2 CI/CD and Environment Promotion . . . 20

3.3 Environment Parity . . . 21

3.4 Scaling and Failure Behaviour . . . 22

4 Data Design **23**

4.1 Data Ownership . . . 23
4.2 Entity–Relationship Model: Identity, Verification, Auction and Bidding . . . 24
4.2.1 Key Design Points . . . 24
4.2.2 Indexes . . . 25
4.3 Entity–Relationship Model: Orders, Escrow, Disputes and Audit . . . 26
4.3.1 The Escrow Ledger . . . 26
4.3.2 Invariants . . . 27
4.3.3 Dispute and Fraud Linkage . . . 27
4.4 Cache and Key Design . . . 27

5 Component and Class Design 29
5.1 Bid Processing . . . 29
5.1.1 The Hot Path . . . 30
5.1.2 Rejection Taxonomy . . . 30
5.2 Payment, Escrow and Dispute Management . . . 31
5.2.1 The Payment Port . . . 31
5.2.2 Saga Steps and Compensations . . . 32
5.2.3 Idempotency . . . 32
5.2.4 Dispute Coupling . . . 32
5.3 Domain Model . . . 33
5.4 Fraud Detection . . . 34
5.4.1 Streaming, Not Batch . . . 34
5.4.2 Signals . . . 35
5.4.3 Measurement Before Tuning . . . 35

6 Behavioural Design 36
6.1 Auction Lifecycle . . . 36
6.2 Order, Escrow and Dispute Settlement . . . 38
6.3 Seller Verification . . . 39
6.4 Sequence: Placing a Bid . . . 40
6.5 Sequence: Settlement Saga . . . 41
6.6 Sequence: Protection Window, Dispute and Release . . . 42
6.7 Activity: Gated Listing and Bid Admission . . . 43

7 REST API Contracts 44
7.1 Contract Principles . . . 44

7.1.1 Error Envelope . . . 44
7.2 Endpoint Inventory . . . 45
7.3 Contract: Bid Submission . . . 47
7.4 Contract: Fee Disclosure . . . 48
7.5 Contract: Dispute Submission . . . 49
7.6 Contract: Administrator Adjudication . . . 50
7.7 Contract: Search . . . 51
7.8 WebSocket Protocol . . . 51
7.8.1 Reconnect and Backpressure . . . 52
7.9 Event Schemas . . . 53
7.10 Contract Testing . . . 54

8 UI/UX Design — Final Mockups 55
8.1 Design Tokens and the Component Library . . . 55
8.2 Auction Browse and Search . . . 56
8.3 Auction Detail and Live Bidding . . . 57
8.4 Seller KYC Submission . . . 58
8.5 Seller Dashboard . . . 59
8.6 Buyer Dispute Submission . . . 60
8.7 Administrator Dispute Adjudication . . . 61
8.8 Administrator KYC Review and Moderation . . . 62
8.9 Notification Tray, Login and Registration . . . 63
8.10 Responsive Layout and Accessibility . . . 64
8.11 Screen-to-Requirement Map . . . 65

9 Cross-Cutting Design Concerns 66
9.1 Security and Privacy . . . 66
9.2 Observability . . . 66
9.3 Performance Budgets . . . 67
9.4 Testing Design . . . 67

10 Traceability 69
10.1 Requirement to Design Element . . . 69
10.2 Design Coverage Check . . . 71

List of Figures

2.1 System context: primary actors and external system dependencies 12

2.2 Container view: thirteen deployable services, their datastores, and the paths between them 13

2.3 Service interaction map: blocking REST calls versus event-driven Kafka flows, with contract-tested boundaries marked 14

2.4 Kafka topic topology, partitioning keys and versioned event schemas 16

3.1 Deployment topology across Vercel, Render, external SaaS and the dedicated load-test host 19

3.2 CI/CD gates and environment promotion 20

4.1 Table ownership per service and the cross-service consistency strategy 23

4.2 Entity–relationship model (1 of 2): users, KYC, auctions and bids 24

4.3 Entity–relationship model (2 of 2): orders, escrow ledger, sagas, disputes, fraud and audit 26

5.1 Component diagram: Bid Processing service, its internal components and its provided and required interfaces 29

5.2 Component diagram: Payment/Escrow and Dispute Management, the saga steps and the three release triggers 31

5.3 Class diagram: aggregate roots, entities and value objects of the core domain . . . 33

5.4 Class diagram: streaming consumer, feature extraction, scoring, ring clustering and the evaluation harness 34

6.1 State machine: auction lifecycle with transition guards 36

6.2 State machine: order settlement, escrow, dispute and the terminal release or refund 38

6.3 State machine: seller KYC verification with audited transitions 39

6.4 Sequence: placing a bid on a contended auction, from client submission to watcher fan-out 40

6.5 Sequence: auction close through invoice, capture and escrow hold, with compensations 41

6.6 Sequence: the three release triggers and the dispute adjudication path 42
6.7 Activity: the two admission-control points every write path passes through 43

8.1 Design tokens and the shared component library 55
8.2 Final mockup: auction browse and search with faceted filters 56
8.3 Final mockup: auction detail and live bid screen with fee disclosure and WebSocket bid history 57
8.4 Final mockup: seller KYC document submission with verification-state feedback . 58
8.5 Final mockup: seller dashboard showing listings, sales, escrow state and expected payout date 59
8.6 Final mockup: window-gated dispute submission with evidence upload 60
8.7 Final mockup: administrator dispute adjudication console 61
8.8 Final mockup: administrator KYC review queue, document viewer and moderation actions 62
8.9 Final mockup: notification tray, sign-in and registration 63
8.10 Final mockup: responsive breakpoints and the accessibility pass 64

List of Tables

1.1 Baseline decisions and their structural consequences . . . 9

2.1 Design drivers and the structures that answer them . . . 11
2.2 Circuit-breaker fallbacks on the three synchronous boundaries . . . 15
2.3 Topic configuration . . . 16
2.4 Architecture decision summary (**FR-DOC-2**–**FR-DOC-5**) . . . 17

3.1 The four environments and what each is authoritative for . . . 21
3.2 Scaling profile and failure behaviour per service . . . 22

4.1 Non-trivial indexes and why each exists . . . 25
4.2 Ledger entry types and their sign convention . . . 27
4.3 Redis key design . . . 28

5.1 Bid rejection taxonomy . . . 30
5.2 Settlement saga steps and their compensating actions (**FR-PAY-6**, **NFR-REL-3**) 32
5.3 Feature signals fed to the shill-bidding scorer . . . 35

6.1 Auction lifecycle transition table . . . 37
6.2 Order settlement transition table . . . 38
6.3 Fee treatment on refund (**FR-PAY-9**) . . . 39

7.1 Public and internal REST endpoints by owning service . . . 45
7.2 WebSocket message types . . . 52
7.3 Contract-tested boundaries . . . 54

8.1 User-facing screens and the requirements each realises . . . 65

9.1 Security controls and where each is implemented . . . 66
9.2 Instrumentation that exists to prove a specific requirement . . . 67

9.3 Latency budget allocation on the bid acknowledgement path (target p95 500 ms) . 67
9.4 Test rigs and the design elements they exercise 68

10.1 Traceability from SRS requirement groups to the design elements that realise them 69
10.2 Design artefact inventory 71

Chapter 1

Introduction

1.1 Purpose

This System Design Document (SDD) specifies *how* the ZeoAuction platform is built. The Software Requirements Specification (SRS) states what the system must do; this document translates each of those requirements into an architecture, a database schema, a set of components and classes, a set of interaction and state models, a published REST API contract, and a final UI/UX design.

It is written to be sufficient for implementation: a developer joining the project should be able to build their assigned service from this document plus the OpenAPI contract, without needing to ask how their service fits into the whole.

1.2 Scope

The design covers the thirteen functional services that make up the platform, together with the two cross-cutting concerns of integration testing and documentation. It specifies:

- architectural views: context, container, interaction, event topology, deployment and CI/CD (Chapters 2–3);
- data design: ownership boundaries, two entity–relationship models, constraints and invariants, and cache key design (Chapter 4);
- component and class design for the four most technically dense services (Chapter 5);
- behavioural design: three state machines, three sequence models and one activity model (Chapter 6);
- REST API contracts in OpenAPI 3.1 form, the WebSocket protocol, and the Kafka event schemas (Chapter 7);
- final UI/UX mockups for every user-facing surface, plus the design-token and component-library specification (Chapter 8);
- cross-cutting concerns and requirement traceability (Chapters 9–10).

Out of scope: production hardening, regulated payment or escrow operation, real monetary movement, and horizontal multi-broker Kafka. The platform is presented throughout as a functional simulation of escrow, consistent with NFR-SEC-4.

1.3 Design Baseline

Seven platform-level decisions are treated as given by this design. They are stated here rather than derived later, because each one changes the shape of the architecture and not merely its parameters.

Table 1.1: Baseline decisions and their structural consequences

Item	Decision	Impact on this design
Deployment target	Render + Vercel, not Kubernetes	No Kubernetes manifests are produced. The deployment artefact of record is a Render blueprint; Kafka becomes a private service with a persistent disk rather than a StatefulSet (Chapter 3).
Service decomposition	Verification and dispute handling are separate services	KYC & Seller Verification and Dispute Management have their own data, contracts and state machines rather than being folded into neighbouring services (Chapters 4–6).
KYC / verification	Mandatory, service-layer gate	A verification state machine, an encrypted document store and a listing gate enforced below the UI (Sections 4.2, 6.3, 6.7).
Escrow and disputes	Buyer protection is first-class	An append-only escrow ledger, a protection-window policy, a freeze rule and an adjudication path (Sections 4.3, 6.2).
Revenue model	Flat convenience fee per sale	A fee configuration singleton fixed before any settlement code is written, and a pre-bid disclosure endpoint consumed by the bid screen (Sections 7.6, 8.3).
Fraud dataset	500 auctions, 200 rings	The dataset generator is a first-class component built before the detector (Section 5.4).
Performance measurement	Dedicated 8 vCPU / 16 GB host	A fourth environment distinct from local, CI and hosted (Section 3.3).

1.4 Definitions Used in This Document

Terms carry the meanings given in SRS Section 1.4. Three additional design-level terms are used here:

`server_seq` The authoritative per-auction sequence number allocated by the bid service before persistence. It, and not a client timestamp or a database insertion order, determines bid ordering.

`Compensating action` The named inverse of a saga step, executed when a later step fails. A compensation always writes a new ledger row; it never deletes one.

`Protection window`
: The configurable interval (default seven days) that opens when the seller marks an item shipped and during which a buyer may raise a dispute.

1.5 Document Conventions

Requirement identifiers (**FR-BID-3**, **NFR-PERF-1**, **AC-08**) refer to the Software Requirements Specification. Identifiers in `monospace` name concrete artefacts — tables, columns, states, error codes and endpoints — and are the exact names used in the implementation.

Chapter 2

Architectural Overview

2.1 Design Drivers

Six forces shape the architecture. Each maps to a specific structural decision rather than to a general aspiration.

Table 2.1: Design drivers and the structures that answer them

Driver	Structural response	Requirement
Write contention on the single hot row (current price)	Redis-buffered current price with compare-and-set and batched persistence; the durable row is written asynchronously in batches	**FR-BID-5**, **NFR-SCALE-2**
Ambiguous ordering of simultaneous equal bids	Server-side sequence allocation before persistence; the ordering is a property of the service, not of the clock or the database	**FR-BID-3**, **AC-04**
Notification latency from client polling	WebSocket gateway with room-based fan-out driven by a Kafka consumer group, replacing polling entirely	**FR-WS-2**, **NFR-PERF-4**
Whole-system failure from one subsystem	Independently deployable services, circuit breakers with declared fallbacks, and per-service health checks	**NFR-REL-1**, **NFR-REL-2**
Fraud discovered after settlement	A streaming consumer on the same ordered bid topic as the bid pipeline, scoring in flight and alerting before auction close	**FR-FRAUD-3**, **FR-FRAUD-4**
Consistency across services without distributed locking	Saga coordination with an explicit compensating action per money-moving step, plus idempotency keys on every such step	**FR-PAY-6**, **NFR-REL-3**

2.2 System Context

Figure 2.1 places ZeoAuction among its actors and external dependencies. Four human or programmatic actor classes drive the system — seller, buyer, administrator and third-party integrator

— and two automated system actors (the fraud detector and the escrow sweep) trigger business events with no human in the loop.

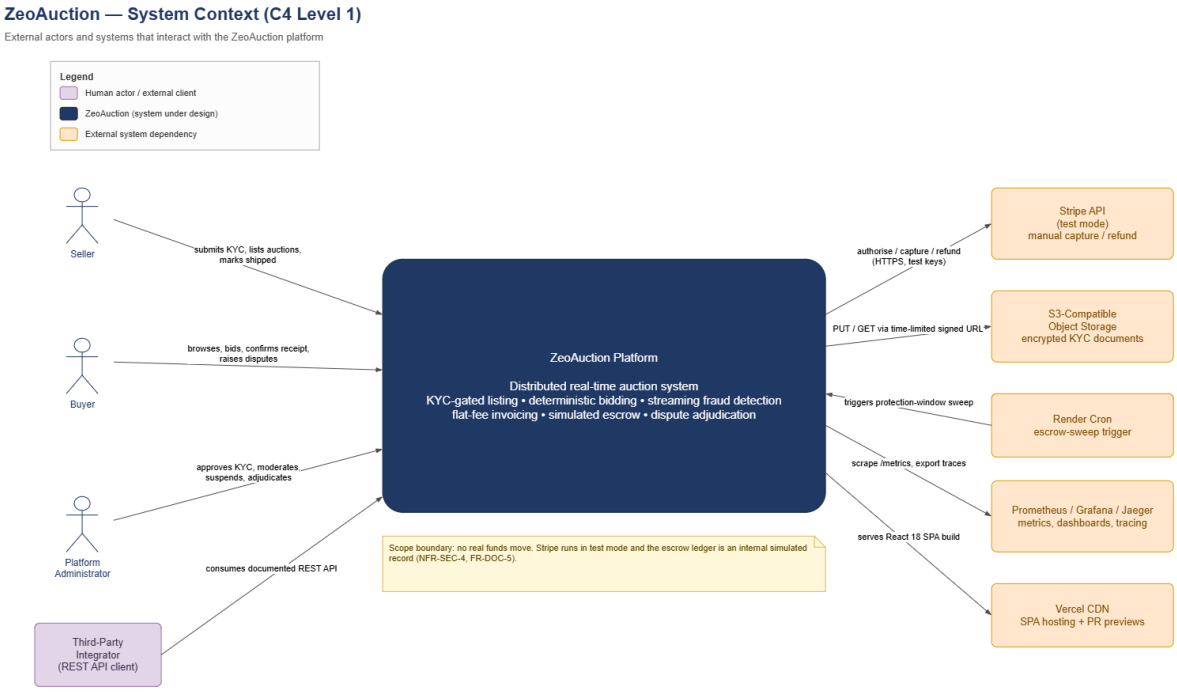


Figure 2.1: System context: primary actors and external system dependencies

The external dependency set is deliberately small. Stripe runs in test mode only; object storage holds KYC documents and dispute evidence; the observability stack is scrape-and-export rather than an inline dependency; and Vercel serves a static build. None of these sits on the critical path of placing a bid, which is the latency- sensitive operation.

2.3 Container Architecture

Figure 2.2 is the primary architectural view: it shows every deployable unit, the datastore each owns, and the synchronous and asynchronous paths between them.

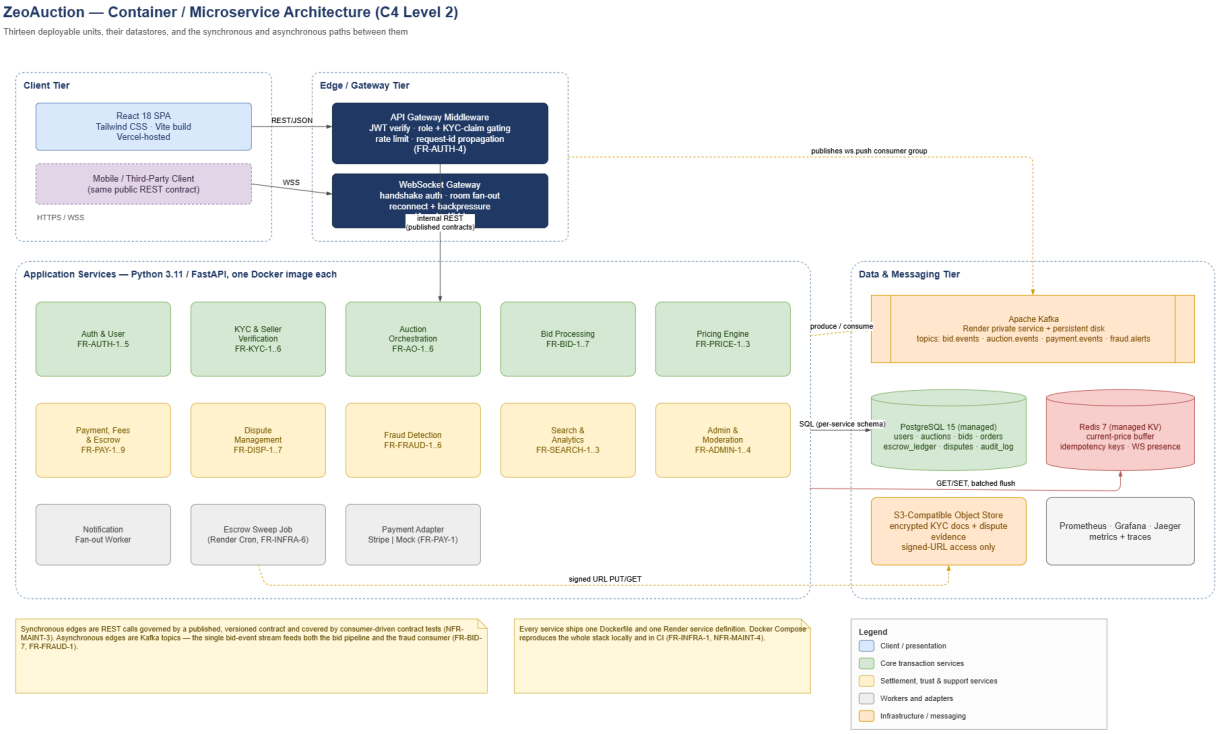


Figure 2.2: Container view: thirteen deployable services, their datastores, and the paths between them

2.3.1 Tiers

Client tier

A React 18 single-page application built with Vite and styled with Tailwind, served from Vercel. The same public REST contract is available to mobile and third-party clients, so the SPA holds no privileged endpoint.

Edge tier

Two components. The API gateway middleware is the single enforcement point for authentication, role gating and verification-status gating (**FR-AUTH-4**); placing it once avoids eleven partial reimplementations. The WebSocket gateway owns connection lifecycle, room membership and backpressure.

Application tier

Eleven FastAPI services plus three workers. Each is a separate Docker image with its own Render service definition, so one can be scaled or restarted without touching the others (**NFR-SCALE-1**).

Data and messaging tier

PostgreSQL 15 for anything monetary or historical, Redis 7 for the hot price path and idempotency keys, Kafka for the ordered replayable event log, and S3-compatible storage for documents.

2.3.2 Why These Service Boundaries

The boundaries follow transactional ownership rather than noun categories. Bid Processing and Auction Orchestration are separate because they have different consistency requirements: bidding is high-frequency and eventually persisted, while lifecycle transitions are low-frequency and strictly guarded. Payment and Dispute are separate because a dispute is a long-lived human workflow measured in days, whereas a settlement saga completes in seconds; merging them would tie a slow workflow to a fast one. KYC is separate because it holds regulated-shaped personal data with its own retention and access rules, and isolating it keeps that data out of every other service's blast radius.

*A useful test of a boundary is whether the two sides can fail independently without corrupting each other. Stopping Fraud Detection must not stop bidding; stopping Dispute Management must not stop settlement of undisputed orders; stopping Search must not stop anything at all. Each of those is verified by a documented chaos test (**NFR-REL-2**, **AC-13**).*

2.4 Synchronous and Asynchronous Interaction

Figure 2.3 separates the two integration styles, because they carry different failure semantics and different testing obligations.

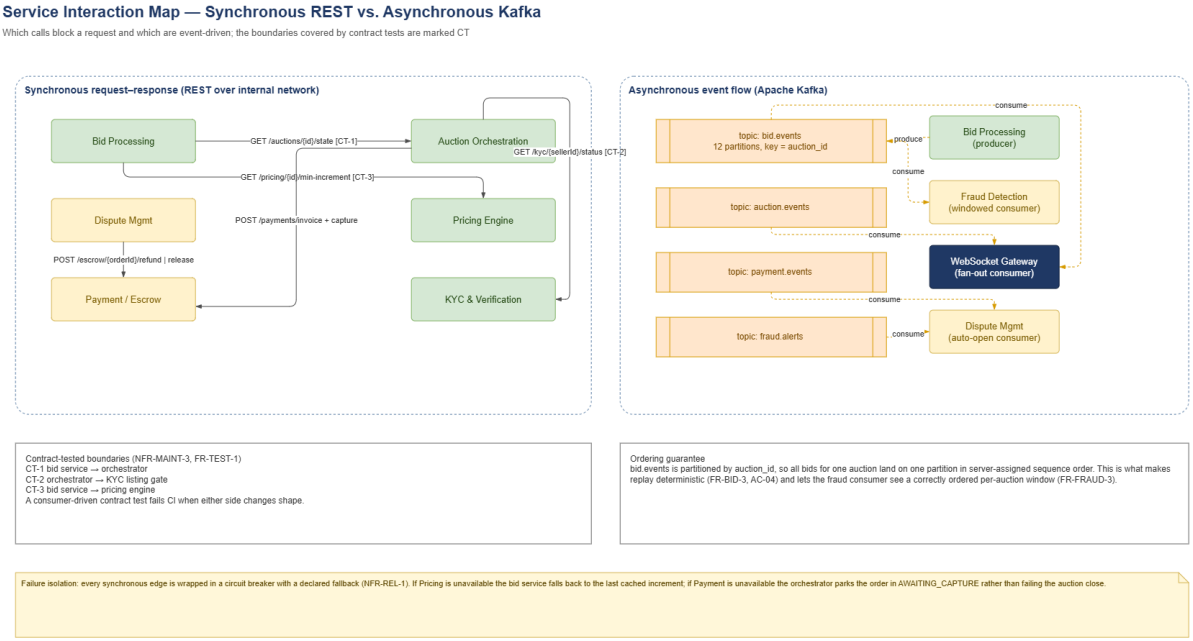


Figure 2.3: Service interaction map: blocking REST calls versus event-driven Kafka flows, with contract-tested boundaries marked

Synchronous calls are used only where a caller genuinely cannot proceed without the answer: bid validation needs the current auction state and the current minimum increment; the listing gate needs verification status. There are exactly three such boundaries, and all three are covered by consumer-driven contract tests that fail CI when either side of the published shape changes

(**NFR-MAINT-3**, **FR-TEST-1**).

Everything else is asynchronous. Fan-out to watchers, fraud scoring, search indexing, dispute auto-opening and ledger projection are all consumer groups on Kafka topics. This is what allows the fraud detector to be stopped, redeployed and restarted from a retained offset without a single bid being lost.

Table 2.2: Circuit-breaker fallbacks on the three synchronous boundaries

Boundary	Breaker opens when	Declared fallback
Bid → Pricing Engine	5 failures in 10 s	Use the last cached minimum increment for that auction; flag the bid batch as `increment_stale` for later audit. Bidding continues.
Bid → Auction Orchestration	5 failures in 10 s	Use the cached auction state if it is under 2 s old; otherwise reject with `AUCTION_STATE_UNAVAILABLE` rather than guess. Bids are never accepted against an unknown state.
Orchestration → Payment	3 failures in 30 s	Park the order in `AWAITING_CAPTURE_RETRY` and retry with the same idempotency key. The auction still closes; settlement is deferred, never abandoned.

2.5 Event and Topic Topology

Figure 2.4 specifies the four topics, their partitioning keys, their producers and their consumer groups, together with the versioned event schemas.

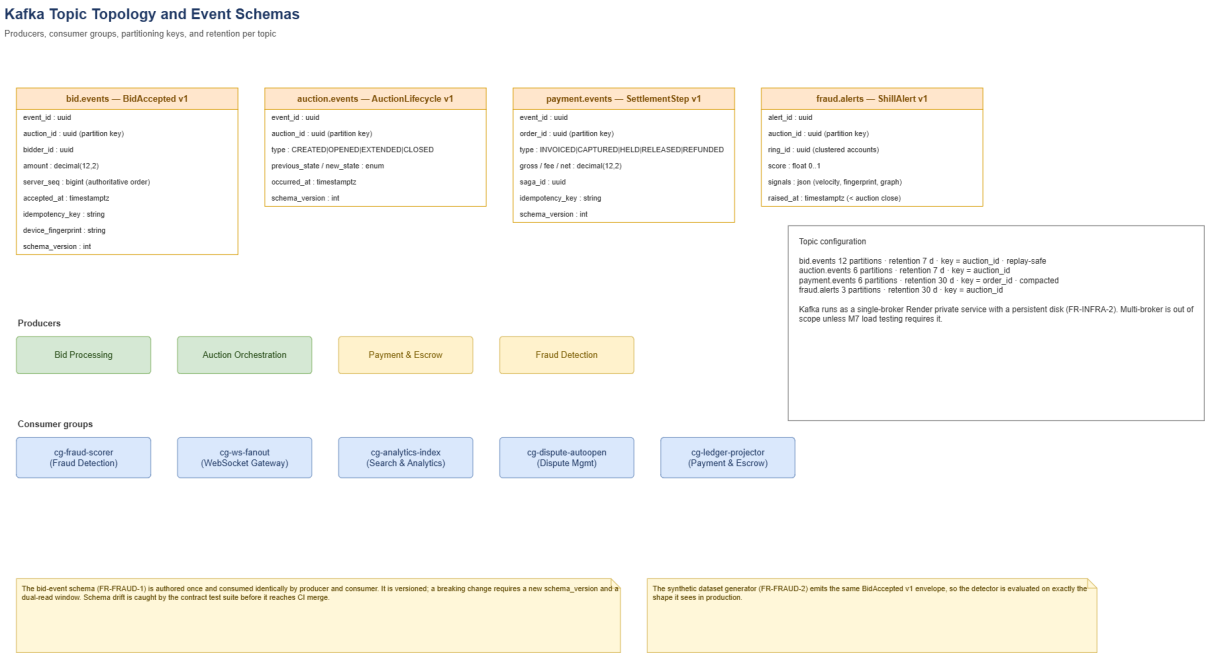


Figure 2.4: Kafka topic topology, partitioning keys and versioned event schemas

The single most important property here is that `bid.events` is partitioned by `auction_id`. All bids for one auction therefore land on one partition in server-assigned sequence order. Two things follow. First, replaying the log reproduces an identical winner, which is what makes **AC-04** testable rather than aspirational. Second, the fraud consumer sees a correctly ordered per-auction window without needing to re-sort, which is what makes in-flight scoring cheap enough to finish before the auction closes.

Table 2.3: Topic configuration

Topic	Parts.	Key	Retention	Rationale
`bid.events`	12	`auction_id`	7 days	Highest volume; partition count sized for the 1000 bids/s aggregate target with headroom.
`auction.events`	6	`auction_id`	7 days	Lifecycle transitions; low volume, but ordering per auction matters.
`payment.events`	6	`order_id`	30 days, compacted	Settlement audit trail; compaction keeps the latest state per order for projection rebuilds.
`fraud.alerts`	3	`auction_id`	30 days	Low volume; retained long enough to correlate with adjudicated disputes.

2.6 Architecture Decision Record Summary

Eight decisions are recorded as architecture decision records. They are summarised here so the design can be read without the full records to hand; the records themselves carry the rejected

alternatives and the supporting evidence.

Table 2.4: Architecture decision summary (**FR-DOC-2**–**FR-DOC-5**)

ID	Decision	Chief alternative rejected	Reason
ADR-1	Server-allocated `server_seq` determines bid order	Database insertion order with `SELECT FOR UPDATE`	Row locking on the hot auction row serialises the whole auction and collapses throughput; sequence allocation keeps ordering deterministic without holding a lock across the write.
ADR-2	Redis compare-and-set for current price, batched persistence	Write-through to PostgreSQL on every bid	A single hot row cannot absorb 200 bids/s; batching moves the durable write off the acknowledgement path while CAS preserves correctness.
ADR-3	Saga with compensations for settlement	Two-phase commit across services	2PC needs a coordinator that can block all participants; compensations degrade to a recorded, auditable reversal instead of a stall.
ADR-4	Escrow ledger is append-only	Mutable balance column on `orders`	An append-only ledger makes reconciliation a sum rather than a trust exercise, and makes double-release detectable rather than invisible.
ADR-5	KYC status carried as a JWT claim	Synchronous lookup on every gated call	Removes a network hop from the listing path; the authoritative record stays in the KYC service and a status change forces token refresh.
ADR-6	Escrow sweep as a platform cron job	In-process scheduler thread	An idle or restarted instance silently skips an in-process timer, which would breach **AC-12** without any error surfacing.

ID	Decision	Chief alternative rejected	Reason
ADR-7	Payment behind a port with a mock adapter	Direct Stripe SDK calls in the service	Keeps the demo runnable with no network and no keys, and confines test-mode restrictions to one adapter.
ADR-8	Fraud dataset built before the detector	Label real traffic after the fact	A target invented after tuning is not a measurement; building the labelled set first makes recall and precision honest numbers.

Chapter 3

Deployment Architecture

3.1 Deployment Topology

Figure 3.1 shows the four environments and the hosted topology.

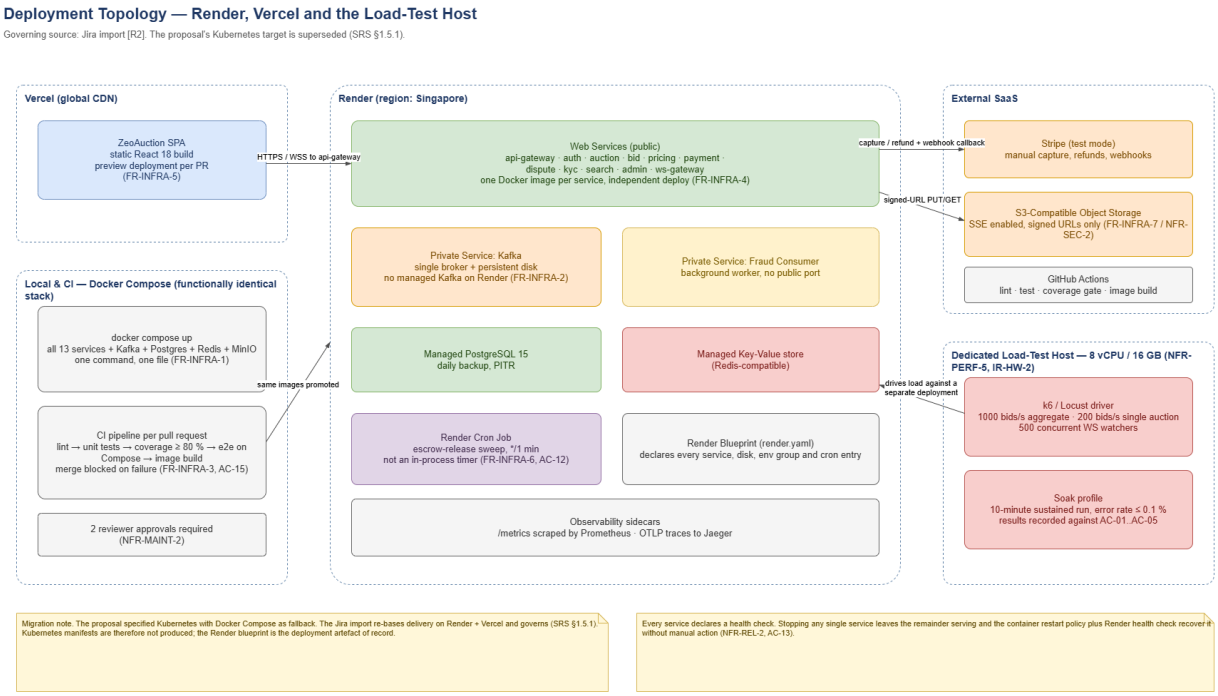


Figure 3.1: Deployment topology across Vercel, Render, external SaaS and the dedicated load-test host

3.1.1 Render

Each backend service is deployed independently from its own Docker image through a Render blueprint (`render.yaml`), which declares every service, disk, environment group and cron entry in one version-controlled file (**FR-INFRA-4**). Two components are private services with no public port: the Kafka broker, which requires a persistent disk because Render offers no managed Kafka

(**FR-INFRA-2**), and the fraud consumer, which is a background worker with no inbound traffic.

3.1.2 Vercel

The SPA is a static build on a global CDN with a preview deployment generated for every pull request (**FR-INFRA-5**). Because the frontend is static and the API is separate, a frontend regression can never take the API down.

3.1.3 The Load-Test Host

Load and latency figures are measured only on a dedicated 8 vCPU / 16 GB host that is separate from Render and from developer laptops (**NFR-PERF-5**, **IR-HW-2**). Figures obtained on a free hosting tier are recorded for interest but are never used for acceptance, because a shared-CPU tier makes a p95 latency number meaningless.

3.2 CI/CD and Environment Promotion

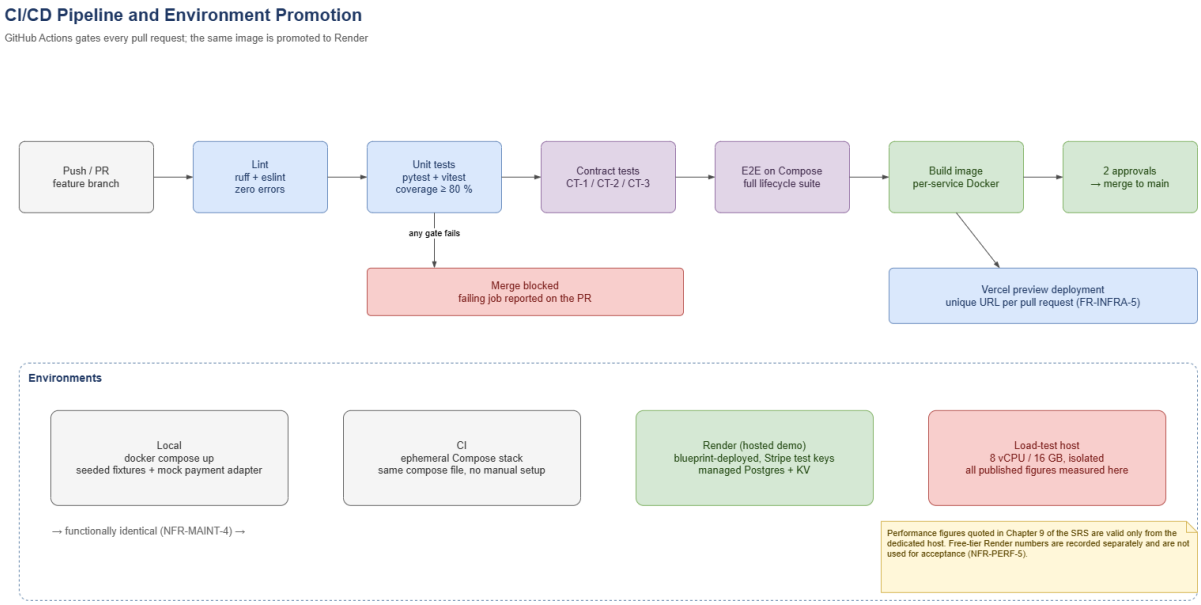


Figure 3.2: CI/CD gates and environment promotion

Every pull request runs the same ordered gate sequence: lint, unit tests with an 80% coverage floor, contract tests on the three published boundaries, an end-to-end run against the Compose stack, and a per-service image build. Any failure blocks the merge, and two reviewer approvals are required regardless (**FR-INFRA-3**, **NFR-MAINT-2**, **AC-15**).

3.3 Environment Parity

Table 3.1: The four environments and what each is authoritative for

Environment	Composition	Authoritative for
Local	`docker compose up`: all services, Kafka, PostgreSQL, Redis, MinIO, seeded fixtures, mock payment adapter	Developer iteration and functional demonstration.
CI	The same Compose file, ephemeral, no manual setup	Correctness gates: unit, contract and end-to-end results.
Render	Blueprint-deployed services, Stripe test keys, managed PostgreSQL and Key-Value store	Functional behaviour of the hosted deployment and resilience demonstrations.
Load-test host	Isolated 8 vCPU / 16 GB machine driving the stack	Every published throughput and latency figure (**AC-01**–**AC-05**).

Parity is a design requirement, not a convenience: the same Compose file that a developer runs is the one CI runs, and the images promoted to Render are the images CI built (**NFR-MAINT-4**). Where the hosted environment necessarily differs — managed PostgreSQL instead of a container, a Render private service instead of a Compose service — the difference is confined to connection strings supplied through an environment group.

3.4 Scaling and Failure Behaviour

Table 3.2: Scaling profile and failure behaviour per service

Service	Scaling	Behaviour when stopped
Bid Processing	Horizontal, stateless	Bidding halts; browsing, search, settlement of closed auctions and admin all continue. Recovers from the restart policy with no manual step.
WebSocket Gateway	Horizontal, sticky by room	Clients fall back to a periodic refresh of the detail page; bids still succeed over REST. Reconnect restores the live feed.
Fraud Detection	Single consumer, scalable by partition	Scoring pauses; the consumer group offset is retained and the backlog is processed on restart, so no bid is missed.
Payment & Escrow	Horizontal, idempotent	Closing auctions park in `AWAITING_CAPTURE_RETRY`. No partial settlement is created because each saga step is individually committed and compensated.
Search & Analytics	Horizontal, read-only	Search returns unavailable; nothing else is affected. Projections rebuild from retained events.
Escrow Sweep (cron)	Single instance, idempotent	A missed run is caught by the next minute's run. Overlapping runs are safe because release carries an idempotency key.

Chapter 4

Data Design

4.1 Data Ownership

Every table has exactly one owning service. No service reads another service's tables directly; it goes through the owner's published REST contract or consumes an event.

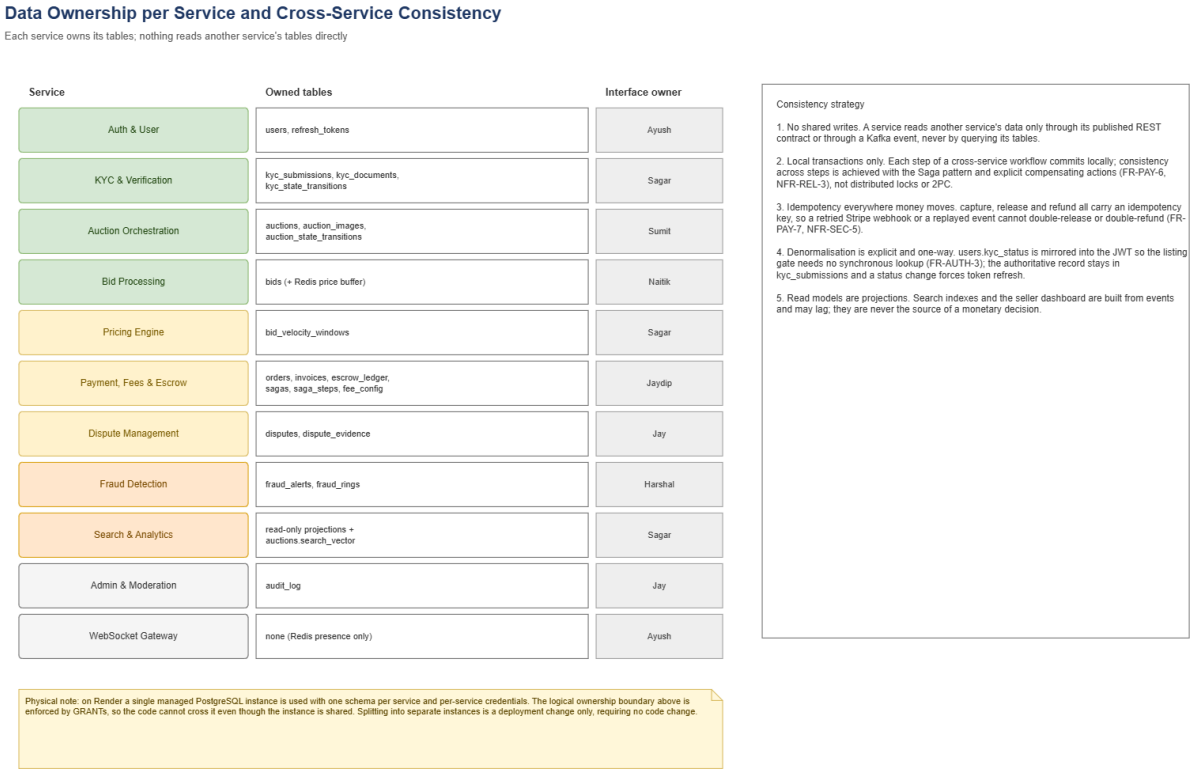


Figure 4.1: Table ownership per service and the cross-service consistency strategy

Physically, the hosted deployment uses one managed PostgreSQL instance with a schema and a distinct credential per service. The logical boundary is therefore enforced by `GRANT`s: the code cannot cross it even though the instance is shared. Splitting into separate instances later is a

deployment change with no code change, which is the point of enforcing the boundary now rather than promising it later.

4.2 Entity–Relationship Model: Identity, Verification, Auction and Bidding

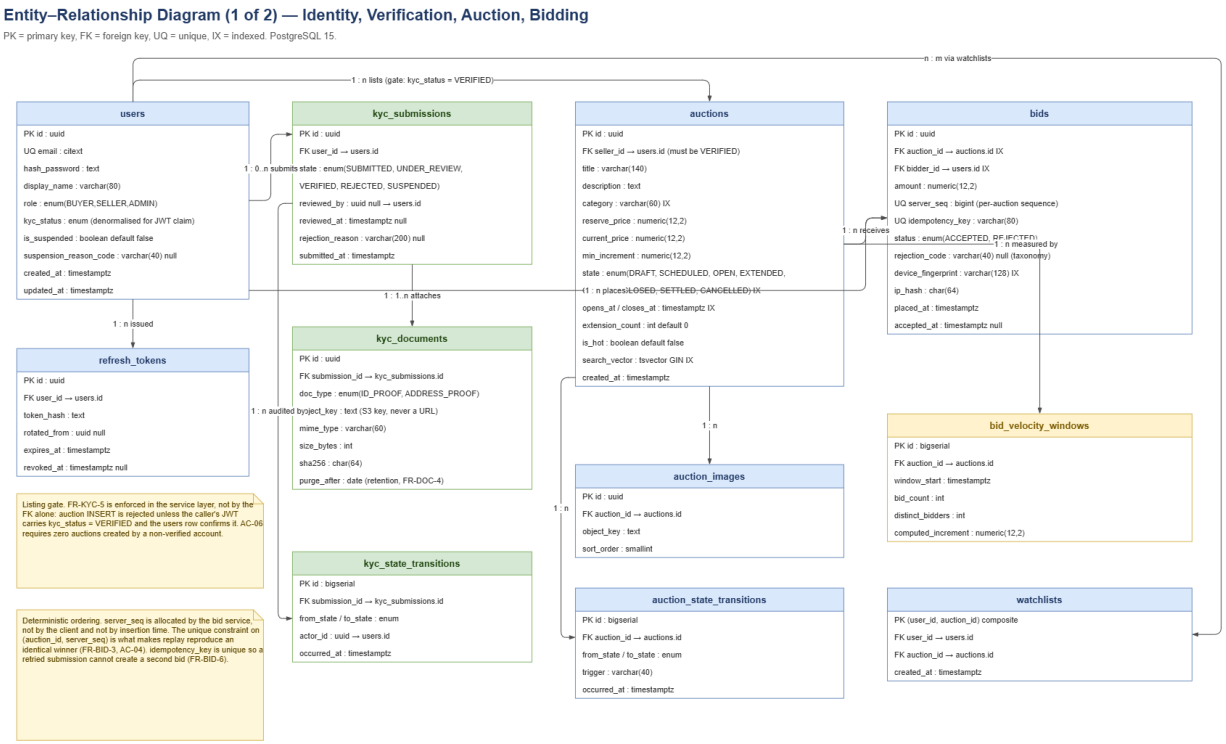


Figure 4.2: Entity–relationship model (1 of 2): users, KYC, auctions and bids

4.2.1 Key Design Points

`users.kyc_status` is denormalised deliberately
The authoritative verification record lives in `kyc_submissions`. The status is mirrored onto `users` so it can be minted into the JWT and so the listing gate needs no synchronous lookup (**FR-AUTH-3**). The mirror is written only by the KYC service, and a status change forces token refresh so a stale `VERIFIED` claim cannot outlive a suspension.

`kyc_documents` stores an object key, never a URL
A stored URL would either be permanent, which defeats time-limiting, or expired, which is useless. The key is resolved to a signed URL at access time with a short TTL (**NFR-SEC-2**). Document bytes never enter application logs, and `purge_after` carries the retention decision recorded in **FR-DOC-4**.

`bids.server_seq` is unique per auction
This single constraint is what makes deterministic conflict resolution enforceable at the storage

layer as well as in code. Two bids cannot occupy the same position in the order, so a replay cannot produce a different winner (**FR-BID-3**).

`bids.idempotency_key` is unique
A retried submission collides on insert and returns the original result rather than creating a second bid (**FR-BID-6**).

`auctions.search_vector` is a stored `tsvector` with a GIN index
Full-text search stays inside PostgreSQL rather than adding a search engine to the dependency set for three filter dimensions (**FR-SEARCH-1**).

`auction_state_transitions` and `kyc_state_transitions` are audit tables
Every transition is recorded with actor and timestamp, so a lifecycle or verification dispute can be answered from data rather than from inference.

4.2.2 Indexes

Table 4.1: Non-trivial indexes and why each exists

Index	Query it serves
`bids (auction_id, server_seq)` unique	Ordering constraint and the bid-history read on the detail page.
`bids (idempotency_key)` unique	Retry collapse on submission.
`bids (device_fingerprint)`	Fraud ring clustering across auctions (**FR-FRAUD-6**).
`auctions (state, closes_at)`	The closing scan and the "ending soonest" sort on the browse screen.
`auctions USING GIN (search_vector)`	Full-text search (**FR-SEARCH-2**).
`orders (protection_window_ends_at)` partial, where state = `SHIPPED`	The escrow sweep's per-minute scan; the partial predicate keeps the index small regardless of historical order volume.
`escrow_ledger (order_id)`	Per-order reconciliation, run 1000 times in **FR-TEST-6**.

4.3 Entity–Relationship Model: Orders, Escrow, Disputes and Audit

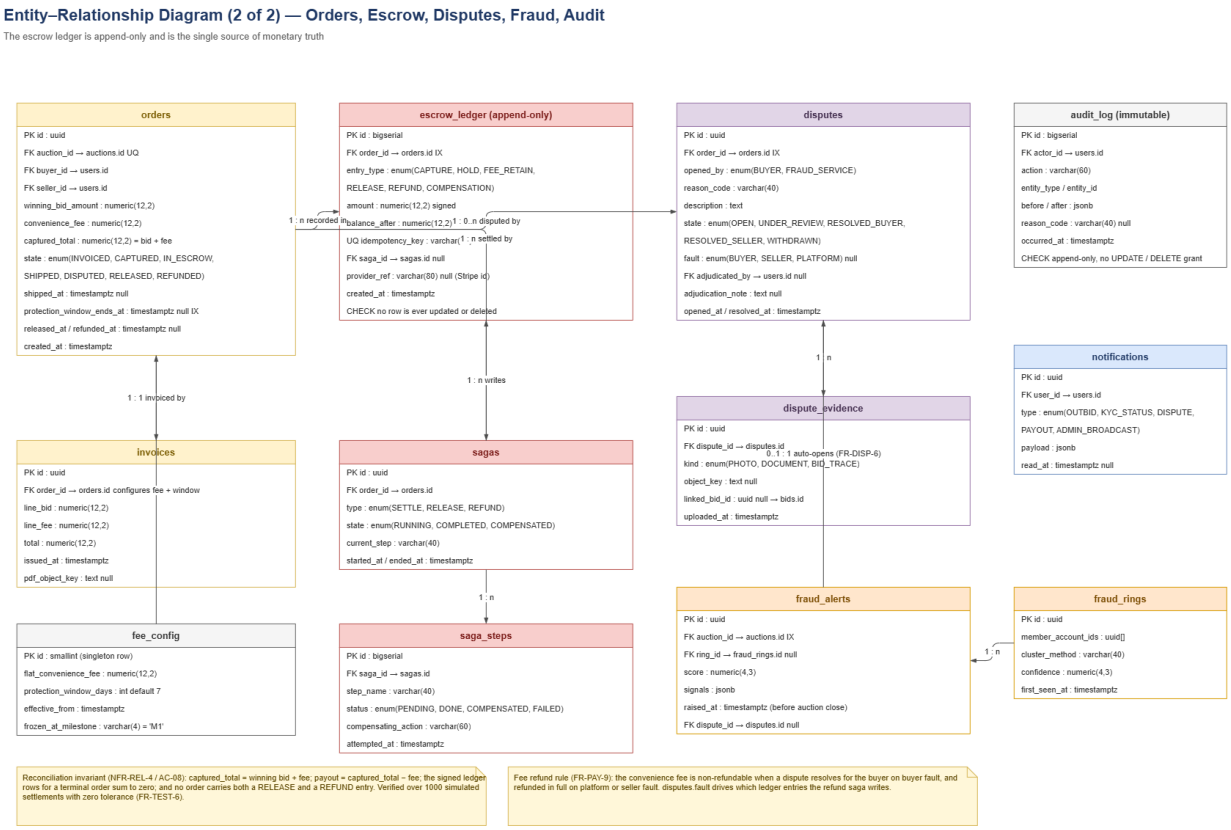


Figure 4.3: Entity–relationship model (2 of 2): orders, escrow ledger, sagas, disputes, fraud and audit

4.3.1 The Escrow Ledger

`escrow_ledger` is the single source of monetary truth and is append-only. Rows are inserted; they are never updated and never deleted. A correction is a new `COMPENSATION` row, not an edit. This has three consequences worth stating explicitly:

1. Reconciliation becomes a sum rather than a comparison of two mutable numbers.
2. A double release becomes visible as two `RELEASE` rows rather than invisible as one overwritten balance.
3. The audit position is defensible: the history of an order cannot be rewritten, only extended.

Table 4.2: Ledger entry types and their sign convention

Entry type	Sign	Meaning
`CAPTURE`	+	Buyer funds captured into platform custody.
`HOLD`	0	Marker that the captured total is under escrow hold; carries the window end date.
`FEE_RETAIN`	−	Flat convenience fee moved to platform revenue.
`RELEASE`	−	Net payout to the seller (captured total minus fee).
`REFUND`	−	Reversal to the buyer; amount depends on fault (**FR-PAY-9**).
`COMPENSATION`	±	Reversal of an earlier entry written by a failed saga step.

4.3.2 Invariants

The following are asserted by the reconciliation test over 1000 simulated settlements, with zero tolerance for a single failure (**NFR-REL-4**, **AC-08**, **FR-TEST-6**):

$$\begin{aligned} \text{captured_total} &= \text{winning_bid_amount} + \text{convenience_fee} \\ \text{seller_payout} &= \text{captured_total} - \text{convenience_fee} \\ \sum_{\text{rows of a terminal order}} \text{amount} &= 0 \\ |\{\text{RELEASE rows}\}| + |\{\text{REFUND rows}\}| &\le 1 \quad \text{per order} \end{aligned}$$

The last of these is the one that matters most in review: no order may ever be both released and refunded. It is enforced by a partial unique index on `escrow_ledger` over `order_id` where `entry_type IN ('RELEASE', 'REFUND')`, so the database refuses the second write even if application logic is wrong.

4.3.3 Dispute and Fraud Linkage

A dispute may be opened by a buyer or by the fraud service. `disputes.opened_by` distinguishes the two, and `fraud_alerts.dispute_id` links an automatically opened case back to the alert that caused it (**FR-DISP-6**). The evidence table accepts three kinds: an uploaded photo, an uploaded document, or a linked bid trace — the last being how a fraud-triggered case presents its evidence to the administrator without asking a human to reconstruct it.

4.4 Cache and Key Design

Redis serves three distinct purposes with three key namespaces. Sharing one instance is acceptable because none of the three is authoritative; all three can be rebuilt.

Table 4.3: Redis key design

Key pattern	TTL	Purpose
`price:{auction_id}`	auction life + 1 h	Current price, updated by compare-and-set on each accepted bid; the hot read for the detail page and for validation (**FR-BID-5**).
`seq:{auction_id}`	auction life + 1 h	Monotonic counter backing `server_seq` allocation.
`idem:{key}`	24 h	Set on first submission with `SETNX`; a retry finds the key and returns the stored result (**FR-BID-6**).
`inc:{auction_id}`	60 s	Cached minimum increment from the pricing engine; also the circuit-breaker fallback value.
`ws:room:{auction_id}`	connection life	WebSocket room membership and watcher count for fan-out.
`flush:pending`	—	Batch buffer of accepted bids awaiting durable write.

The batched write is the only place where a crash could lose data, so the buffer is bounded by both size and age (whichever comes first), and the Kafka publish happens before the buffer flush. Kafka, not Redis, is the recovery source: an unflushed batch is reconstructed by replaying `bid.events` from the last persisted `server_seq`.

Chapter 5

Component and Class Design

This chapter details the four services whose internal structure is not obvious from the container view. The remaining services are conventional controller–service–repository FastAPI applications and are specified by their OpenAPI contracts in Chapter 7.

5.1 Bid Processing

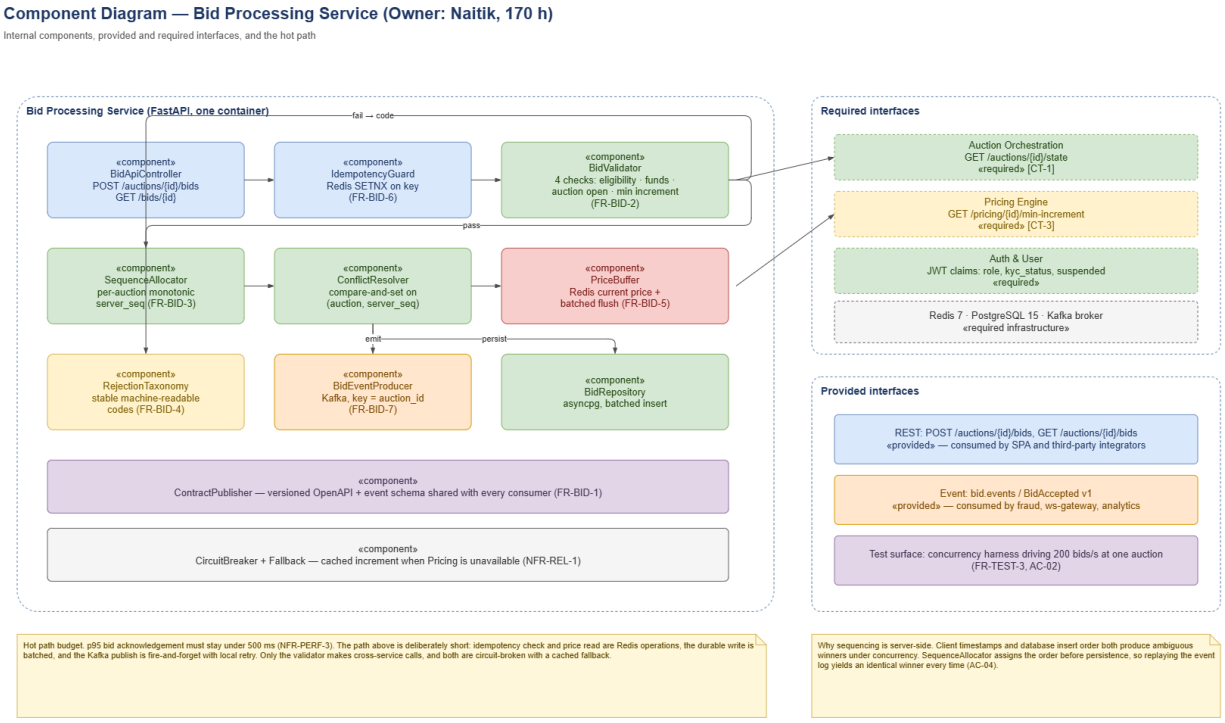


Figure 5.1: Component diagram: Bid Processing service, its internal components and its provided and required interfaces

5.1.1 The Hot Path

The acknowledgement path is deliberately short because it carries a p95 budget of 500 ms (**NFR-PERF-3**) under an aggregate load of 1000 bids/s (**NFR-PERF-1**). In order:

1. `IdempotencyGuard` performs a Redis `SETNX`. A duplicate returns the stored result immediately and the path ends here.
2. `BidValidator` evaluates the four conditions of **FR-BID-2** — account eligibility, sufficient funds, auction open, amount at or above the minimum increment — and rejects before any state change occurs.
3. `SequenceAllocator` allocates `server_seq` from the Redis counter. This fixes the ordering before anything is persisted.
4. `ConflictResolver` performs a compare-and-set on the buffered price. The winner proceeds; the loser is re-validated against the new price.
5. The accepted bid is appended to the write batch and published to `bid.events`.
6. The client is acknowledged. Durable persistence and fan-out both continue after the acknowledgement.

Only step 2 makes cross-service calls, and both are circuit-broken with the fallbacks tabulated in Section 2.4.

5.1.2 Rejection Taxonomy

The rejection codes are a published, stable, machine-readable vocabulary consumed identically by the SPA and by third-party integrators (**FR-BID-4**, **US-22**). They are part of the versioned contract; adding a code is a minor version bump, changing the meaning of one is a breaking change.

Table 5.1: Bid rejection taxonomy

Code	HTTP	Condition
`SELLER_NOT_VERIFIED`	403	Caller's KYC status is not `VERIFIED` (listing path).
`ACCOUNT_SUSPENDED`	403	Caller is suspended by an administrator.
`SELF_BIDDING_FORBIDDEN`	403	Bidder is the seller of the auction.
`AUCTION_NOT_OPEN`	409	Auction state is not `OPEN` or `EXTENDED`.
`AUCTION_CLOSED`	409	Auction closed between validation and resolution.
`BID_BELOW_INCREMENT`	422	Amount is below current price plus minimum increment.
`INSUFFICIENT_FUNDS`	402	Funds check failed.
`DUPLICATE_IDEMPOTENCY_KEY`	200	Retry of a prior submission; the original result is returned.
`AUCTION_STATE_UNAVAILABLE`	503	Orchestrator unreachable and no fresh cached state; bid deliberately not accepted.

5.2 Payment, Escrow and Dispute Management

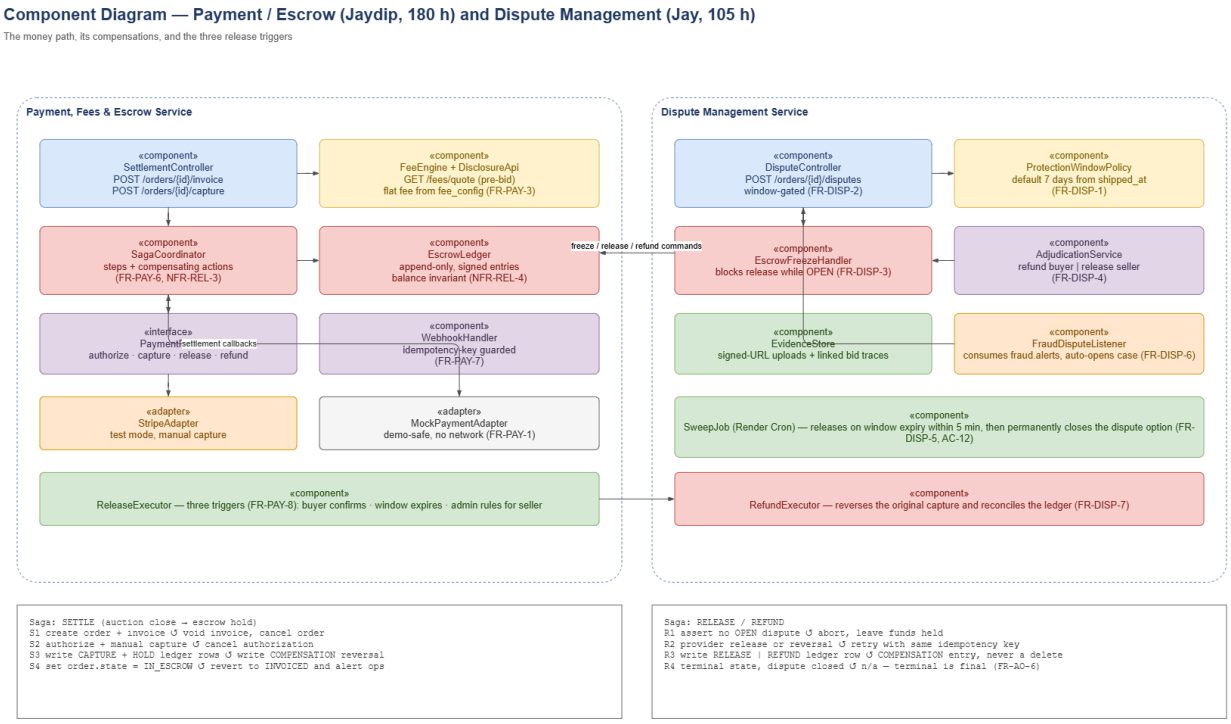


Figure 5.2: Component diagram: Payment/Escrow and Dispute Management, the saga steps and the three release triggers

5.2.1 The Payment Port

Payment capability sits behind `PaymentPort`, an interface with two implementations: `StripeAdapter` (test mode, manual capture) and `MockPaymentAdapter` (no network, deterministic). Stripe is one adapter, not a dependency of the running system (**FR-PAY-1**). This is what allows the platform to be demonstrated offline with no keys, and what confines test-mode restrictions on escrow-style flows to a single class.

Manual capture is what realises the escrow-hold primitive: the payment is authorised and captured into platform custody, and the transfer to the seller happens only through the defined release transition. There is no other code path that moves funds to a seller (**FR-PAY-5**).

5.2.2 Saga Steps and Compensations

Table 5.2: Settlement saga steps and their compensating actions (**FR-PAY-6**, **NFR-REL-3**)

Step	Forward action	Compensating action
S1	Create order and issue invoice (winning bid + flat fee)	Void the invoice and cancel the order.
S2	Authorise and capture through `PaymentPort` (manual capture)	Cancel the authorisation; if already captured, issue a full reversal.
S3	Write `CAPTURE`, `HOLD` and `FEE_RETAIN` ledger rows	Write `COMPENSATION` rows of equal and opposite amount. Nothing is deleted.
S4	Set `orders.state = IN_ESCROW`	Revert to `INVOICED` and raise an operational alert.
R1	Assert no `OPEN` dispute exists for the order	Abort; funds remain held. This is a guard, not a mutation.
R2	Provider release or reversal	Retry with the identical idempotency key; a duplicate is a no-op.
R3	Write `RELEASE` or `REFUND` row	Write `COMPENSATION`; the partial unique index prevents a second terminal row.
R4	Terminal state; dispute option closed	None. Terminal is final (**FR-AO-6**).

5.2.3 Idempotency

Every money-moving operation carries an idempotency key derived from the order and the operation, for example `order:{id}:release`. The key is stored with the ledger row under a unique constraint. A retried Stripe webhook, a duplicated queue delivery and a double-clicked admin button therefore all converge on the same single effect (**FR-PAY-7**, **NFR-SEC-5**).

5.2.4 Dispute Coupling

Dispute Management does not write the ledger. It issues freeze, release and refund commands to Payment across a published contract, and Payment owns the money. This keeps a long-running human workflow from holding a transactional resource, and keeps all ledger writes in one service where the invariants are enforced.

5.3 Domain Model

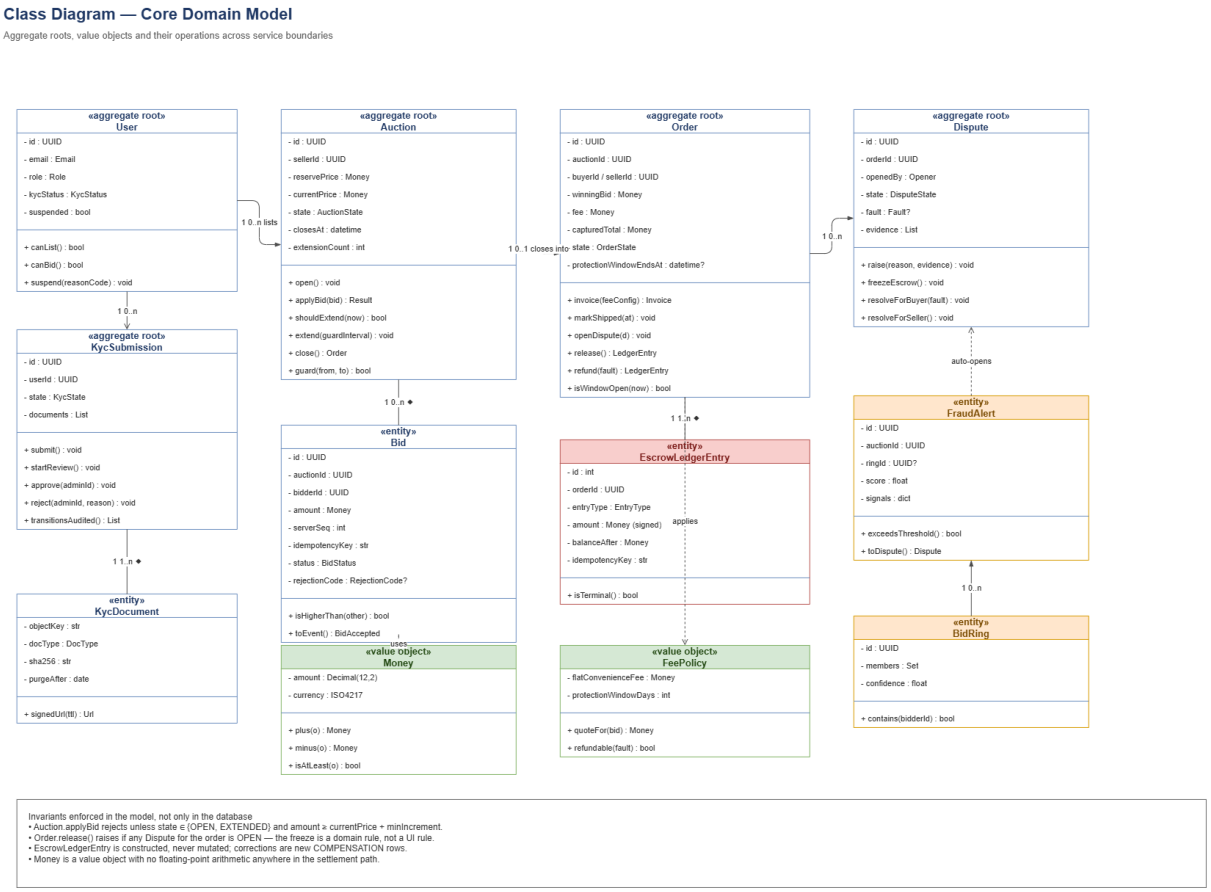


Figure 5.3: Class diagram: aggregate roots, entities and value objects of the core domain

Four aggregate roots carry the business rules: `User`, `Auction`, `Order` and `Dispute`, with `KycSubmission` as a fifth in the verification context. The rules live in the model, not only in the database:

- `Auction.applyBid` refuses unless the state is `OPEN` or `EXTENDED` and the amount clears the current price plus minimum increment.
- `Order.release` raises if any dispute on the order is `OPEN`. The freeze is a domain rule, not a UI rule (**FR-DISP-3**).
- `EscrowLedgerEntry` is constructed and never mutated.
- `Money` is a value object over `Decimal`. No floating-point arithmetic appears anywhere in the settlement path.
- `FeePolicy.refundable(fault)` encodes **FR-PAY-9** in one place, so the administrator selects a fault and the accounting follows automatically.

5.4 Fraud Detection

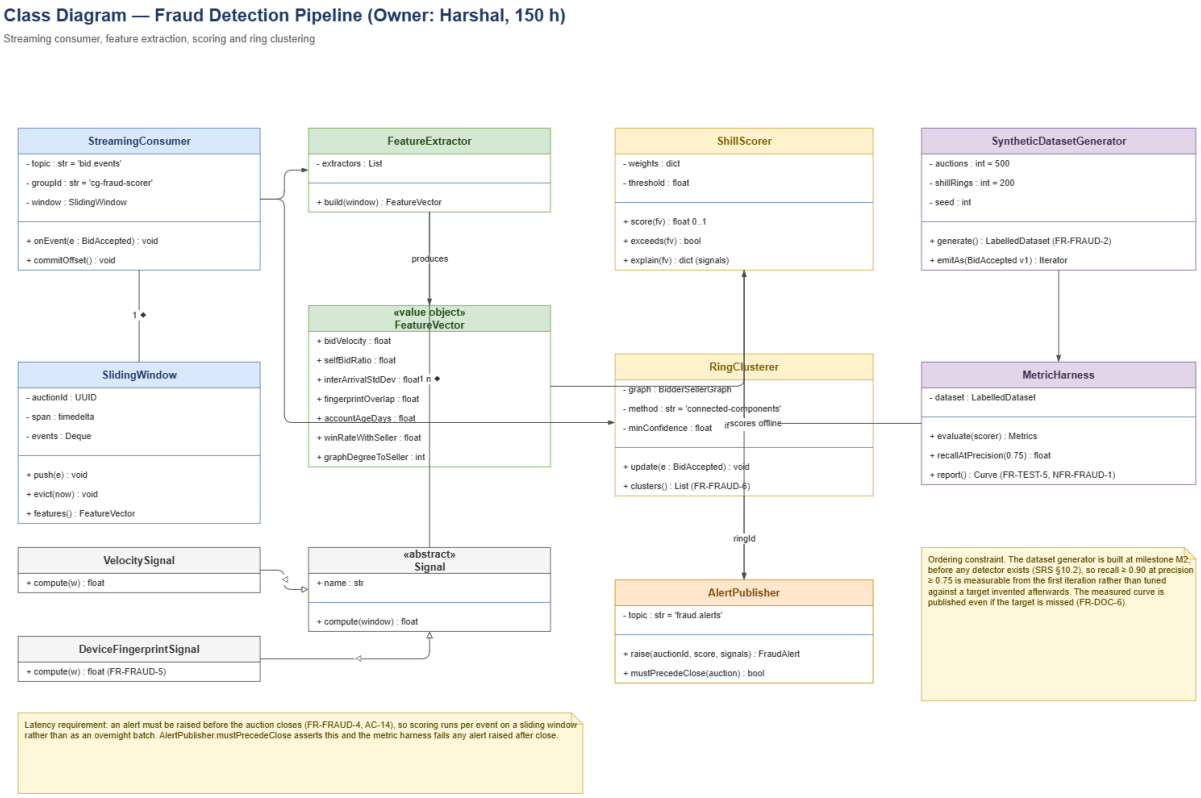


Figure 5.4: Class diagram: streaming consumer, feature extraction, scoring, ring clustering and the evaluation harness

5.4.1 Streaming, Not Batch

`StreamingConsumer` maintains a `SlidingWindow` per auction over the ordered `bid.events` partition and scores on each event. An alert must be raised before the auction closes (**FR-FRAUD-4**, **AC-14**), so `AlertPublisher.mustPrecedeClose` asserts this condition and the evaluation harness counts a late alert as a miss rather than a hit.

5.4.2 Signals

Table 5.3: Feature signals fed to the shill-bidding scorer

Signal	What it captures
`bidVelocity`	Bids per unit time in the window; shill activity clusters.
`selfBidRatio`	Proportion of bids from accounts linked to the seller.
`interArrivalStdDev`	Machine-like regularity in bid timing.
`fingerprintOverlap`	Shared device fingerprints across supposedly distinct bidders (**FR-FRAUD-5**).
`accountAgeDays`	Freshly created accounts appearing only on one seller's auctions.
`winRateWithSeller`	Accounts that repeatedly bid but never win against one seller.
`graphDegreeToSeller`	Degree in the bidder–seller graph used for ring clustering (**FR-FRAUD-6**).

5.4.3 Measurement Before Tuning

`SyntheticDatasetGenerator` produces 500 auctions containing 200 labelled shill-bidding rings and emits them in the same `BidAccepted v1` envelope the detector sees in production (**FR-FRAUD-2**). It is built before any detector exists, so that the target of recall ≥ 0.90 at precision ≥ 0.75 is measurable from the first iteration (**NFR-FRAUD-1**). The measured recall–precision curve is published whether or not the target is met (**FR-DOC-6**).

Chapter 6

Behavioural Design

6.1 Auction Lifecycle

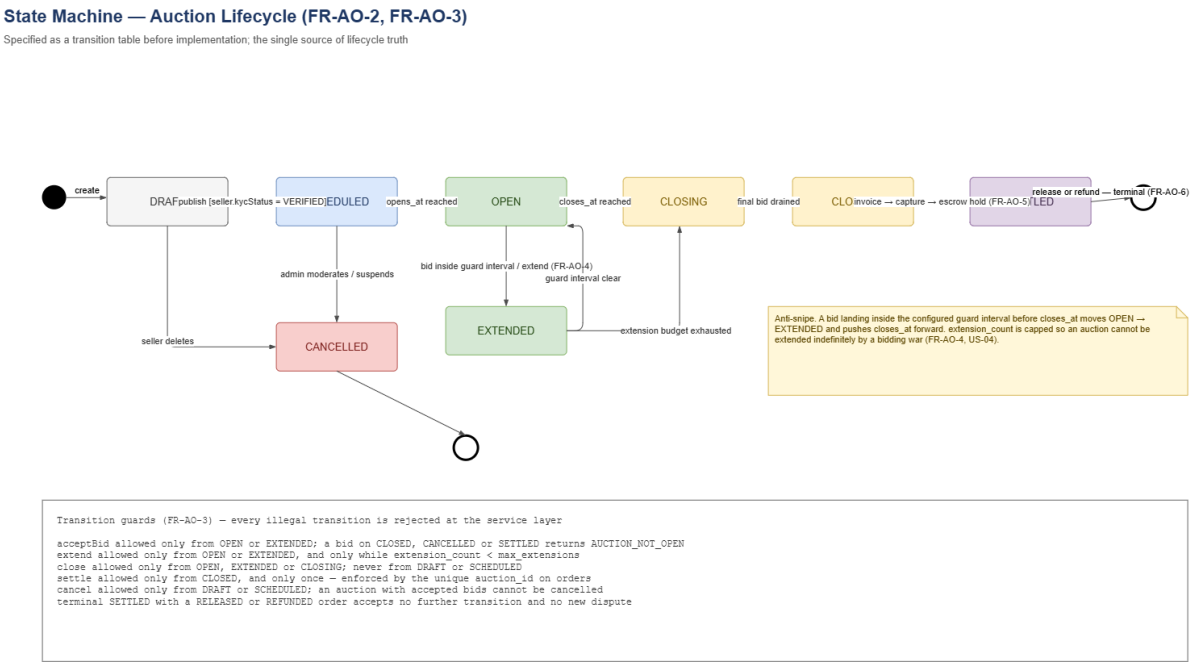


Figure 6.1: State machine: auction lifecycle with transition guards

The lifecycle state machine is specified as a transition table before implementation and is the single source of lifecycle truth (**FR-AO-2**). Illegal transitions are rejected by guards, not merely undefined (**FR-AO-3**).

Table 6.1: Auction lifecycle transition table

Event	From	To	Guard
`publish`	`DRAFT`	`SCHEDULED`	Seller's `kyc_status` is `VERIFIED` and account not suspended.
`open`	`SCHEDULED`	`OPEN`	`opens_at` reached.
`acceptBid`	`OPEN`, `EXTENDED`	unchanged	Bid validated; otherwise `AUCTION_NOT_OPEN`.
`extend`	`OPEN`, `EXTENDED`	`EXTENDED`	Bid inside the guard interval and `extension_count` below the cap.
`settleWindow`	`EXTENDED`	`OPEN`	Guard interval clear.
`close`	`OPEN`, `EXTENDED`, `CLOSING`	`CLOSED`	`closes_at` reached and the bid queue drained.
`settle`	`CLOSED`	`SETTLED`	Exactly once; enforced by the unique `auction_id` on `orders`.
`cancel`	`DRAFT`, `SCHEDULED`	`CANCELLED`	No accepted bids exist.
`any`	`SETTLED` terminal, `CANCELLED`	—	Refused. No transition and no new dispute (**FR-AO-6**).

Anti-snipe extension (**FR-AO-4**) moves `OPEN` to `EXTENDED` and pushes `closes_at` forward when a bid lands inside the configured guard interval. `extension_count` is capped so a bidding war cannot extend an auction indefinitely.

6.2 Order, Escrow and Dispute Settlement

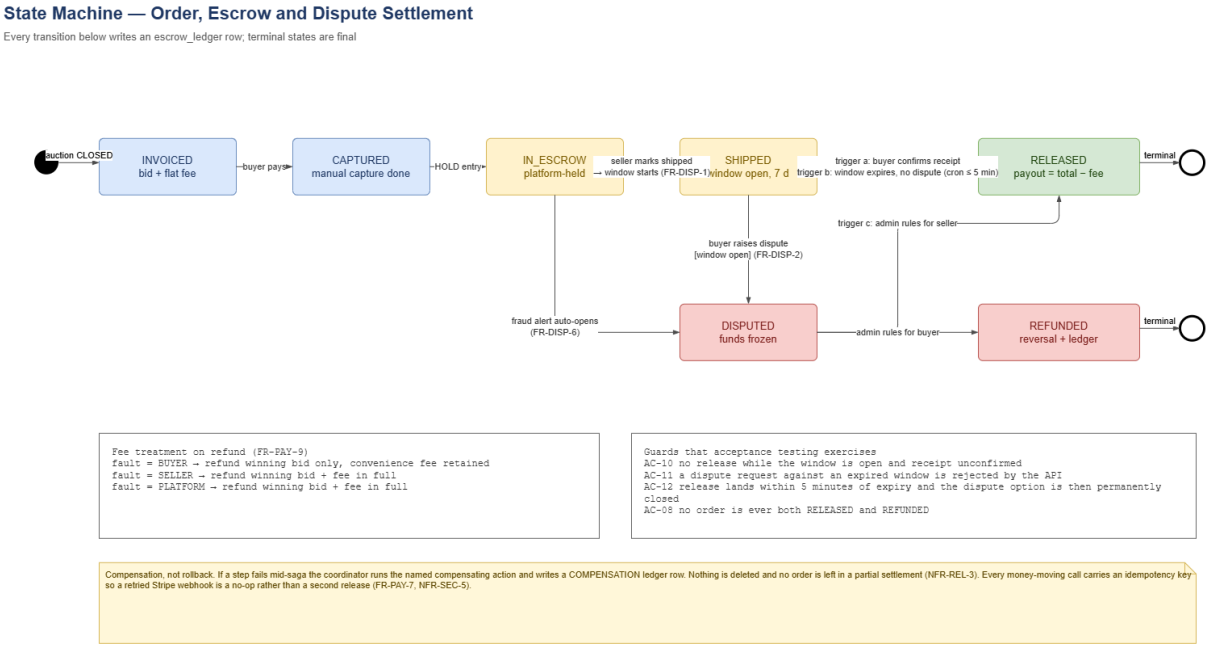


Figure 6.2: State machine: order settlement, escrow, dispute and the terminal release or refund

Table 6.2: Order settlement transition table

Event	From	To	Effect and guard
`invoice`	—	`INVOICED`	Auction `CLOSED`; total is winning bid plus flat fee; losing bidders are never invoiced (**AC-09**).
`capture`	`INVOICED`	`CAPTURED`	Manual capture through `PaymentPort`; writes `CAPTURE`.
`hold`	`CAPTURED`	`IN_ESCROW`	Writes `HOLD` and `FEE_RETAIN`.
`markShipped`	`IN_ESCROW`	`SHIPPED`	Starts the protection window (**FR-DISP-1**).
`raiseDispute`	`SHIPPED`, `IN_ESCROW`	`DISPUTED`	Only while the window is open; freezes escrow (**FR-DISP-2**, **FR-DISP-3**).
`confirmReceipt`	`SHIPPED`	`RELEASED`	Trigger (a): immediate release.
`sweepExpiry`	`SHIPPED`	`RELEASED`	Trigger (b): window expired with no dispute; within 5 minutes (**AC-12**).
`adjudicateSeller`	`DISPUTED`	`RELEASED`	Trigger (c): administrator rules for the seller.
`adjudicateBuyer`	`DISPUTED`	`REFUNDED`	Reversal of the original capture; fee treatment per fault (**FR-PAY-9**).
`any`	`RELEASED`, `REFUNDED`	—	Refused. Terminal; the dispute option is permanently closed.

Table 6.3: Fee treatment on refund (**FR-PAY-9**)

Fault	Buyer receives	Rationale
`BUYER`	Winning bid only; fee retained	The platform performed the service it charged for.
`SELLER`	Winning bid and fee in full	The buyer should not pay a fee for a transaction the seller failed.
`PLATFORM`	Winning bid and fee in full	The platform bears the cost of its own failure.

6.3 Seller Verification

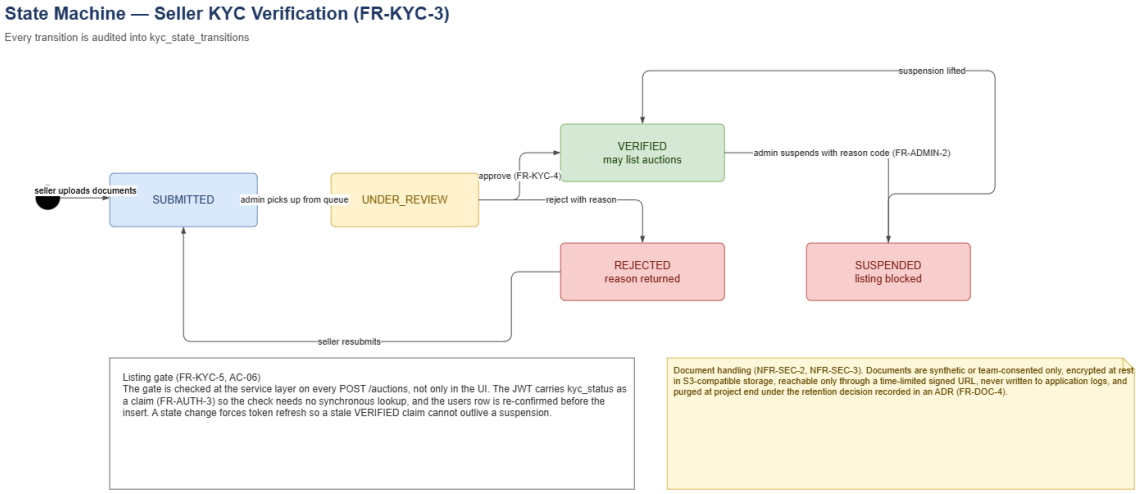


Figure 6.3: State machine: seller KYC verification with audited transitions

Every transition is written to `kyc_state_transitions` with actor and timestamp (**FR-KYC-3**). A rejection returns the administrator's reason and re-opens the submission path, so `REJECTED` is recoverable rather than terminal. Suspension is reachable from `VERIFIED` and is reversible.

6.4 Sequence: Placing a Bid

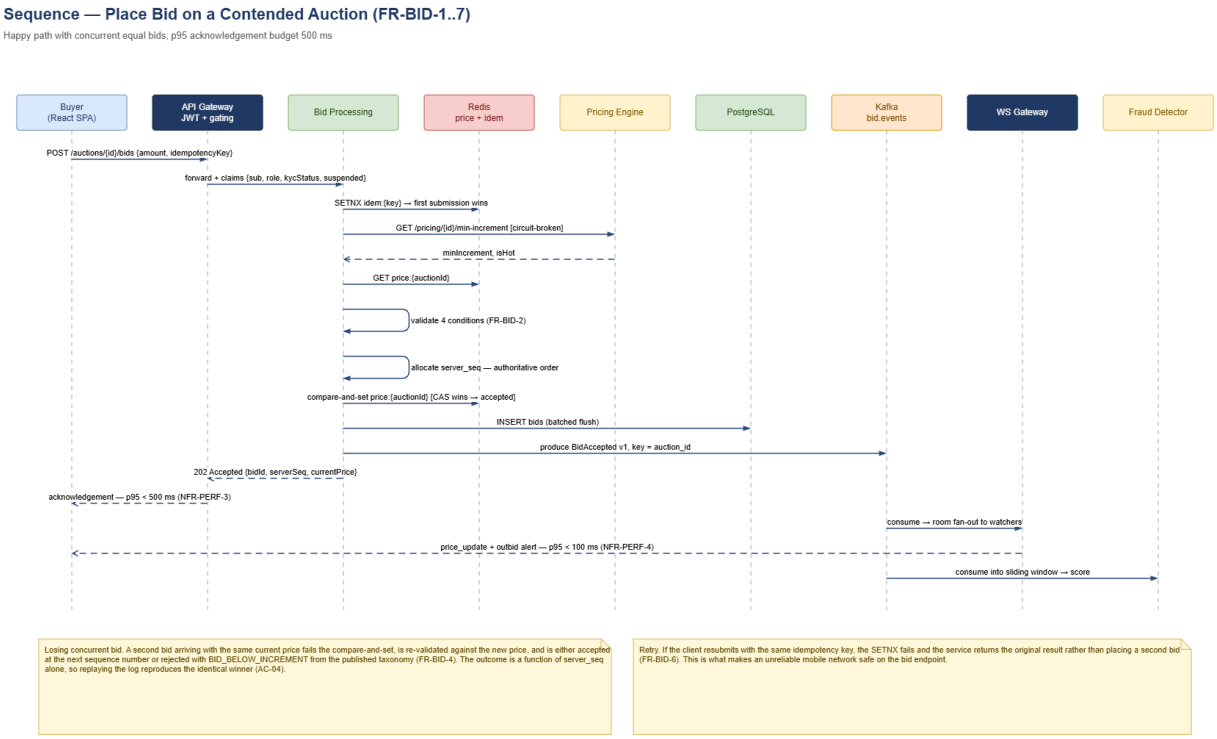


Figure 6.4: Sequence: placing a bid on a contended auction, from client submission to watcher fan-out

Two aspects of this sequence deserve emphasis. First, the client is acknowledged before durable persistence completes; correctness is preserved because ordering was already fixed by `server_seq` and the event was already published to a durable log. Second, the fan-out to watchers is driven by the Kafka consumer, not by the request thread, so a slow watcher cannot slow a bidder — which is what makes the 100 ms p95 fan-out target achievable with 500 concurrent watchers (**NFR-PERF-4**).

A losing concurrent bid fails the compare-and-set, is re-validated against the new price, and is either accepted at the next sequence number or rejected with `BID_BELOW_INCREMENT`. The outcome is a function of `server_seq` alone.

6.5 Sequence: Settlement Saga

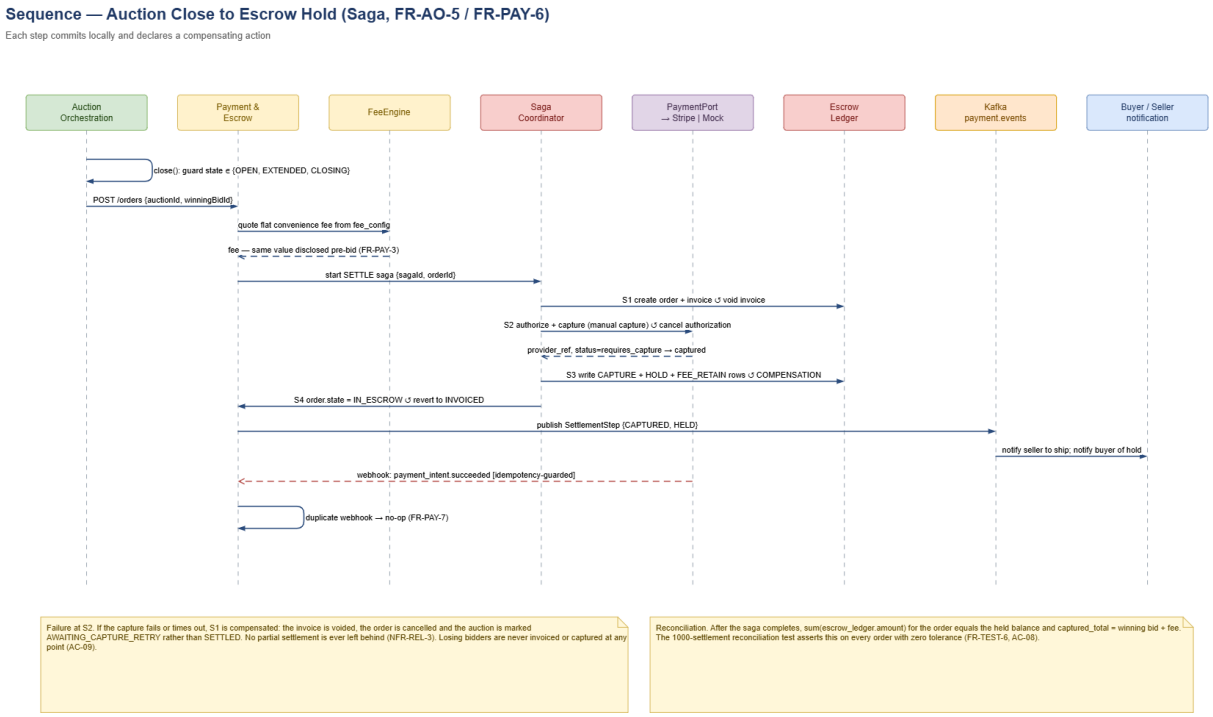


Figure 6.5: Sequence: auction close through invoice, capture and escrow hold, with compensations

If capture fails or times out at S2, S1 is compensated: the invoice is voided, the order is cancelled, and the auction is parked in `AWAITING_CAPTURE_RETRY` rather than being marked `SETTLED`. No partial settlement is left behind (**NFR-REL-3**). Duplicate Stripe webhooks are absorbed by the idempotency guard and become no-ops (**FR-PAY-7**).

6.6 Sequence: Protection Window, Dispute and Release

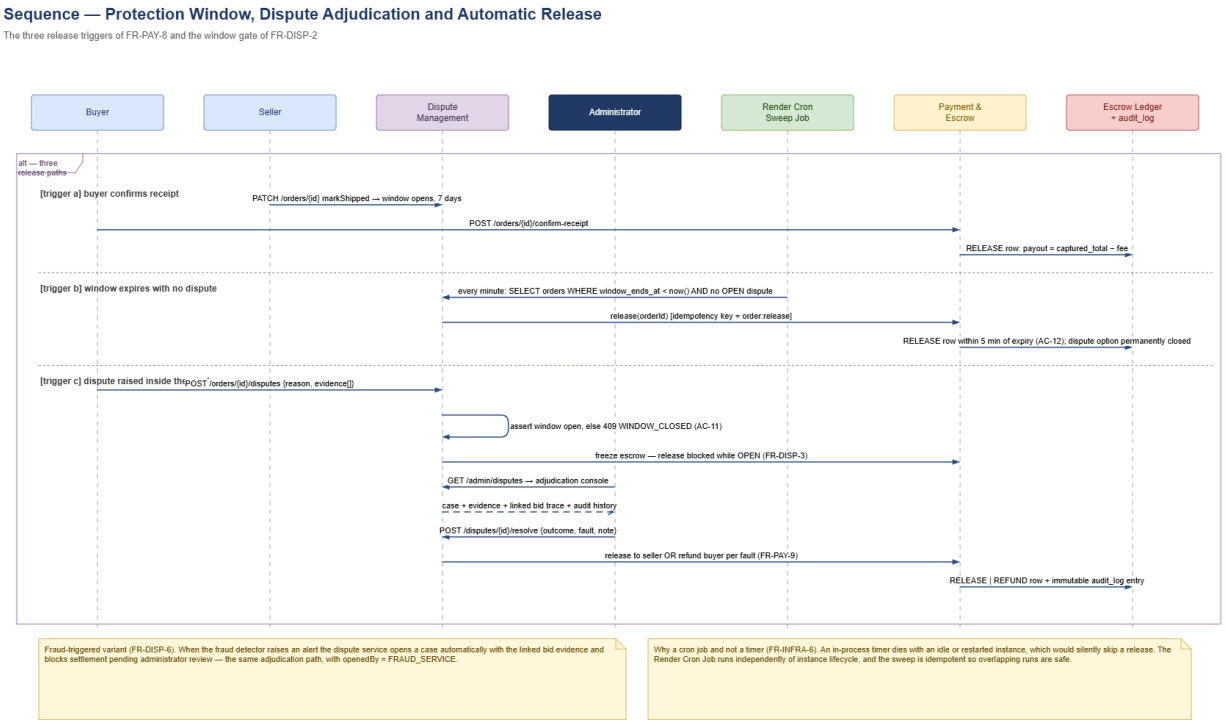


Figure 6.6: Sequence: the three release triggers and the dispute adjudication path

The three release triggers of **FR-PAY-8** appear as alternatives in one sequence because they converge on the same ledger write and the same terminal state. The fraud-triggered variant follows the same adjudication path with `openedBy = FRAUD_SERVICE` (**FR-DISP-6**).

The sweep runs as a platform cron job rather than an in-process timer because an idle or restarted instance would silently skip a release, breaching **AC-12** with no error surfacing anywhere (**FR-INFRA-6**). The sweep is idempotent, so overlapping runs are safe.

6.7 Activity: Gated Listing and Bid Admission

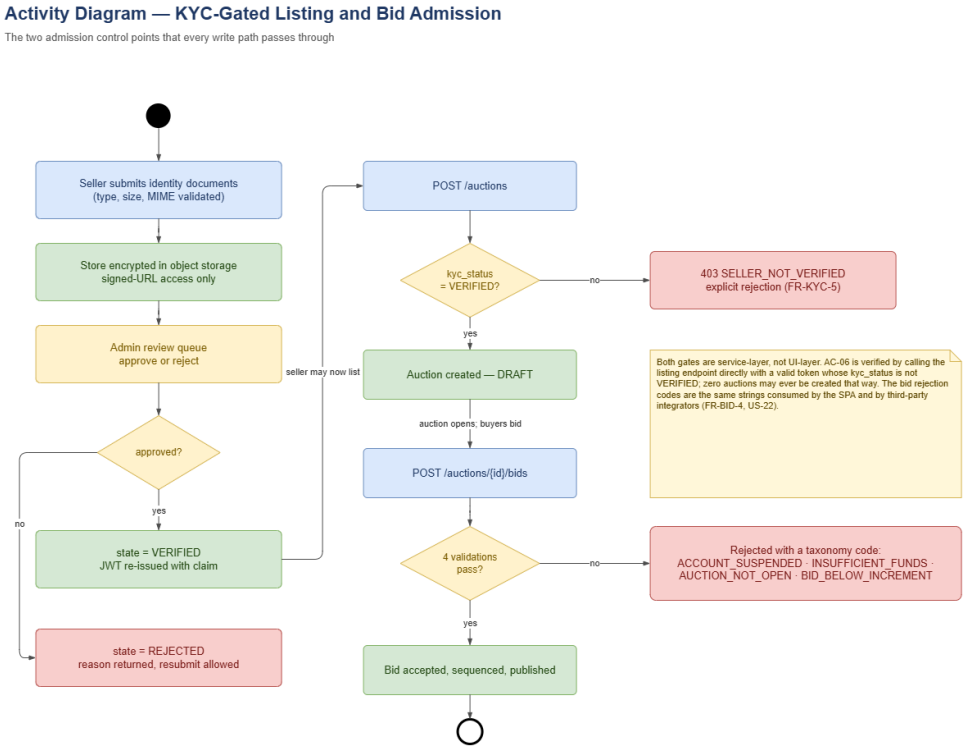


Figure 6.7: Activity: the two admission-control points every write path passes through

Both gates are enforced at the service layer, not in the SPA. **AC-06** is verified by calling the listing endpoint directly with a valid token whose `kyc_status` is not `VERIFIED`; zero auctions may ever be created that way.

Chapter 7

REST API Contracts

7.1 Contract Principles

The API is the product boundary, not an implementation detail. The same surface serves the SPA, mobile clients and third-party integrators (**US-21**), so nothing in it is private to the frontend.

Generated, not hand-written
The specification is generated from FastAPI route signatures and Pydantic models (**FR-DOC-1**). A route and its documentation cannot drift apart, because they are the same declaration.

Versioned by path prefix
All endpoints live under `/api/v1`. A breaking change requires `/api/v2` and a deprecation window; adding an optional field or a new rejection code does not.

Bearer authentication
Every non-public call carries a JWT in the `Authorization` header, encoding `sub`, `role`, `kyc_status` and `suspended` (**IR-COM-3**). Access tokens live 15 minutes; refresh tokens rotate on use (**FR-AUTH-2**).

Idempotency on writes that move money or state
`POST` endpoints that place a bid, capture, release or refund accept an `Idempotency-Key` header and return the original result on replay.

Uniform errors
Every error response uses one envelope with a stable machine-readable `code`. Clients switch on `code`, never on the human message.

Cursor-free pagination
List endpoints take `page` and `page_size` and return `total`. Simplicity is preferred over cursor pagination at this scale.

7.1.1 Error Envelope

```
components:
  schemas:
```

```yaml
    Error:
      type: object
      required: [code, message, request_id]
      properties:
        code:       { type: string, example: BID_BELOW_INCREMENT }
        message:    { type: string, example: "Minimum next bid is 1250.00" }
        details:    { type: object, nullable: true }
        request_id: { type: string, format: uuid }
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

Listing 7.1: Uniform error envelope returned by every endpoint

7.2 Endpoint Inventory

Table 7.1: Public and internal REST endpoints by owning service

Method	Path	Auth	Requirement
Auth & User Service			
POST	`/api/v1/auth/register`	public	FR-AUTH-1
POST	`/api/v1/auth/login`	public	FR-AUTH-1, FR-AUTH-2
POST	`/api/v1/auth/refresh`	refresh token	FR-AUTH-2
POST	`/api/v1/auth/logout`	bearer	FR-AUTH-2
GET	`/api/v1/users/me`	bearer	FR-AUTH-5
PATCH	`/api/v1/users/me`	bearer	FR-AUTH-5
KYC & Seller Verification			
POST	`/api/v1/kyc/submissions`	bearer	FR-KYC-1
POST	`/api/v1/kyc/submissions/{id}/documents`	bearer	FR-KYC-1, FR-KYC-2
GET	`/api/v1/kyc/submissions/me`	bearer	FR-KYC-6
GET	`/api/v1/admin/kyc/queue`	admin	FR-KYC-4
POST	`/api/v1/admin/kyc/{id}/approve`	admin	FR-KYC-4
POST	`/api/v1/admin/kyc/{id}/reject`	admin	FR-KYC-4
Auction Orchestration			
POST	`/api/v1/auctions`	bearer, verified	FR-AO-1, FR-KYC-5
GET	`/api/v1/auctions/{id}`	public	FR-AO-1
GET	`/api/v1/auctions/{id}/state`	internal	FR-AO-2 [CT-1]
POST	`/api/v1/auctions/{id}/publish`	bearer, owner	FR-AO-2
POST	`/api/v1/auctions/{id}/cancel`	bearer, owner	FR-AO-3
Bid Processing			

Method	Path	Auth	Requirement
POST	`/api/v1/auctions/{id}/bids`	bearer	FR-BID-1–7
GET	`/api/v1/auctions/{id}/bids`	public	FR-BID-1
GET	`/api/v1/bids/{id}`	bearer	FR-BID-1
Pricing Engine			
GET	`/api/v1/pricing/{auctionId}/min-increment`	internal	FR-PRICE-2 [CT-3]
GET	`/api/v1/pricing/{auctionId}/velocity`	internal	FR-PRICE-1
Payment, Fees & Escrow			
GET	`/api/v1/fees/quote`	public	FR-PAY-3
GET	`/api/v1/orders/{id}`	bearer, party	FR-PAY-4
GET	`/api/v1/orders/{id}/invoice`	bearer, party	FR-PAY-4
POST	`/api/v1/orders/{id}/capture`	internal	FR-PAY-2
POST	`/api/v1/orders/{id}/confirm-receipt`	bearer, buyer	FR-PAY-8
POST	`/api/v1/orders/{id}/ship`	bearer, seller	FR-DISP-1
POST	`/api/v1/orders/{id}/release`	internal	FR-PAY-8
POST	`/api/v1/orders/{id}/refund`	internal	FR-DISP-7
POST	`/api/v1/webhooks/stripe`	signature	FR-PAY-7
Dispute Management			
POST	`/api/v1/orders/{id}/disputes`	bearer, buyer	FR-DISP-2
GET	`/api/v1/disputes/{id}`	bearer, party	FR-DISP-4
POST	`/api/v1/disputes/{id}/evidence`	bearer, party	FR-DISP-2
GET	`/api/v1/admin/disputes`	admin	FR-DISP-4
POST	`/api/v1/admin/disputes/{id}/resolve`	admin	FR-DISP-4, FR-PAY-9
Search & Analytics			
GET	`/api/v1/search/auctions`	public	FR-SEARCH-1, FR-SEARCH-2
GET	`/api/v1/sellers/me/dashboard`	bearer, seller	FR-SEARCH-3
Admin & Moderation			
GET	`/api/v1/admin/listings/queue`	admin	FR-ADMIN-1
POST	`/api/v1/admin/users/{id}/suspend`	admin	FR-ADMIN-2
GET	`/api/v1/admin/audit-log`	admin	FR-ADMIN-3
GET	`/api/v1/admin/metrics/daily`	admin	FR-ADMIN-4
POST	`/api/v1/admin/broadcasts`	admin	FR-ADMIN-4
Platform			
GET	`/api/v1/notifications`	bearer	FR-UI-4
GET	`/healthz`, `/readyz`	public	NFR-REL-2
GET	`/metrics`	internal	NFR-OBS-1

7.3 Contract: Bid Submission

This is the most heavily specified endpoint in the system because it carries the throughput target, the idempotency guarantee and the published rejection taxonomy.

```yaml
paths:
  /api/v1/auctions/{auctionId}/bids:
    post:
      operationId: placeBid
      tags: [bids]
      summary: Place a bid on an open auction
      security: [{ bearerAuth: [] }]
      parameters:
        - name: auctionId
          in: path
          required: true
          schema: { type: string, format: uuid }
        - name: Idempotency-Key
          in: header
          required: true
          description: >
            Client-generated key, unique per logical submission. A retry with the
            same key returns the original result and does not place a second bid.
          schema: { type: string, maxLength: 80 }
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [amount]
              properties:
                amount:
                  type: string
                  description: Decimal string; never a float.
                  example: "1250.00"
                device_fingerprint:
                  type: string
                  description: Opaque client signal consumed by fraud scoring.
      responses:
        "202":
          description: Bid accepted and sequenced.
          content:
            application/json:
              schema:
                type: object
                required: [bid_id, server_seq, current_price, accepted_at]
                properties:
                  bid_id:        { type: string, format: uuid }
                  server_seq:    { type: integer, format: int64 }
                  current_price: { type: string, example: "1250.00" }
                  min_next_bid:  { type: string, example: "1275.00" }
                  accepted_at:   { type: string, format: date-time }
        "402": { $ref: "#/components/responses/InsufficientFunds" }
        "403": { $ref: "#/components/responses/Forbidden" }
        "409": { $ref: "#/components/responses/AuctionNotOpen" }
        "422": { $ref: "#/components/responses/BidBelowIncrement" }
        "503": { $ref: "#/components/responses/StateUnavailable" }
```

Listing 7.2: `POST /api/v1/auctions/{auctionId}/bids` — bid submission (**FR-BID-1**, **FR-BID-2**, **FR-BID-4**, **FR-BID-6**)

7.4 Contract: Fee Disclosure

The fee quote endpoint exists so that the bid screen can disclose the flat convenience fee before a bid is placed, which is what **AC-07** verifies. The identical value is used at invoice time; the quote is not an estimate.

```yaml
/api/v1/fees/quote:
  get:
    operationId: quoteFee
    tags: [payments]
    summary: Flat convenience fee and resulting total for a prospective bid
    parameters:
      - name: bid_amount
        in: query
        required: true
        schema: { type: string, example: "1250.00" }
    responses:
      "200":
        content:
          application/json:
            schema:
              type: object
              required: [bid_amount, convenience_fee, total, fee_version]
              properties:
                bid_amount:      { type: string, example: "1250.00" }
                convenience_fee: { type: string, example: "25.00" }
                total:           { type: string, example: "1275.00" }
                currency:        { type: string, example: "USD" }
                fee_version:
                  type: string
                  description: >
                    Identifies the fee_config row in force. Fixed before any
                    settlement code is written, so disclosure, invoice and
                    settlement cannot disagree.
```

Listing 7.3: `GET /api/v1/fees/quote` — pre-bid fee disclosure (**FR-PAY-3**, **NFR-USE-3**)

7.5 Contract: Dispute Submission

The window gate is enforced by the API, independently of the UI. A request against a closed window is refused with `WINDOW_CLOSED`, which is what **AC-11** verifies.

```yaml
/api/v1/orders/{orderId}/disputes:
  post:
    operationId: raiseDispute
    tags: [disputes]
    security: [{ bearerAuth: [] }]
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [reason_code, description]
            properties:
              reason_code:
                type: string
                enum: [ITEM_NOT_RECEIVED, NOT_AS_DESCRIBED, DAMAGED_IN_TRANSIT,
                       COUNTERFEIT, OTHER]
              description: { type: string, maxLength: 4000 }
              evidence_object_keys:
                type: array
                items: { type: string }
    responses:
      "201":
        description: Dispute opened; escrow frozen for this order.
        content:
          application/json:
            schema:
              type: object
              properties:
                dispute_id:   { type: string, format: uuid }
                state:        { type: string, example: OPEN }
                escrow_state: { type: string, example: FROZEN }
      "409":
        description: >
          WINDOW_CLOSED - the protection window has expired and the order is
          terminal. Verified by acceptance criterion AC-11.
        content:
          application/json:
            schema: { $ref: "#/components/schemas/Error" }
```

Listing 7.4: `POST /api/v1/orders/{orderId}/disputes` — window-gated dispute (**FR-DISP-2**)

7.6 Contract: Administrator Adjudication

Fault selection drives the fee outcome automatically; the administrator chooses the fault, not the accounting (**FR-PAY-9**).

```yaml
/api/v1/admin/disputes/{disputeId}/resolve:
  post:
    operationId: resolveDispute
    tags: [admin, disputes]
    security: [{ bearerAuth: [] }]
    parameters:
      - name: Idempotency-Key
        in: header
        required: true
        schema: { type: string }
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [outcome, note]
            properties:
              outcome: { type: string, enum: [RELEASE_TO_SELLER, REFUND_BUYER] }
              fault:
                type: string
                enum: [BUYER, SELLER, PLATFORM]
                description: >
                  Required when outcome is REFUND_BUYER. SELLER or PLATFORM refunds
                  the convenience fee in full; BUYER retains it (FR-PAY-9).
              note:
                type: string
                description: Written to the immutable audit log (FR-ADMIN-3).
    responses:
      "200":
        content:
          application/json:
            schema:
              type: object
              properties:
                dispute_state: { type: string, example: RESOLVED_BUYER }
                order_state:   { type: string, example: REFUNDED }
                ledger_entry_id: { type: integer }
                fee_refunded:  { type: boolean }
```

Listing 7.5: `POST /api/v1/admin/disputes/{id}/resolve` — adjudication (**FR-DISP-4**)

7.7 Contract: Search

```yaml
/api/v1/search/auctions:
  get:
    operationId: searchAuctions
    tags: [search]
    parameters:
      - { name: q,          in: query, schema: { type: string } }
      - { name: category,   in: query, schema: { type: string } }
      - { name: price_min,  in: query, schema: { type: string } }
      - { name: price_max,  in: query, schema: { type: string } }
      - { name: status,     in: query,
          schema: { type: string, enum: [OPEN, SCHEDULED, CLOSED] } }
      - { name: sort,       in: query,
          schema: { type: string, enum: [ending_soonest, newest, price_asc,
                                         price_desc], default: ending_soonest } }
      - { name: page,       in: query, schema: { type: integer, default: 1 } }
      - { name: page_size,  in: query,
          schema: { type: integer, default: 20, maximum: 100 } }
    responses:
      "200":
        content:
          application/json:
            schema:
              type: object
              required: [items, page, page_size, total]
              properties:
                items:
                  type: array
                  items: { $ref: "#/components/schemas/AuctionSummary" }
                page:      { type: integer }
                page_size: { type: integer }
                total:     { type: integer }
```

Listing 7.6: `GET /api/v1/search/auctions` — paginated search and filter (**FR-SEARCH-2**)

7.8 WebSocket Protocol

The WebSocket channel is a separate protocol surface with its own contract. It is authenticated at handshake, room-scoped, and carries no command traffic — clients place bids over REST and receive updates over the socket (**FR-WS-1**, **FR-WS-2**, **IR-COM-2**).

Table 7.2: WebSocket message types

Dir.	Type	Payload and semantics
C→S	`subscribe`	`{auction_id}`. Joins the auction room. Rejected if the token is expired.
C→S	`unsubscribe`	`{auction_id}`. Leaves the room.
C→S	`ping`	Keepalive; the server replies `pong`.
S→C	`price_update`	`{auction_id, current_price, min_next_bid, server_seq}`. Clients discard any message whose `server_seq` is not greater than the last seen, which makes out-of-order delivery harmless.
S→C	`outbid`	`{auction_id, your_bid_id, current_price}`. Sent only to the displaced bidder (**US-10**).
S→C	`auction_extended`	`{auction_id, new_closes_at}`. Anti-snipe extension.
S→C	`auction_closed`	`{auction_id, winning_bid_id}`.
S→C	`notification`	KYC status, dispute and payout events for the connected user's tray (**FR-UI-4**).

7.8.1 Reconnect and Backpressure

On reconnect the client sends `subscribe` with its last seen `server_seq`; the gateway replies with the current price snapshot so no update is lost across the gap (**FR-WS-3**). If a client's send buffer exceeds its bound, the gateway drops intermediate `price_update` messages and sends only the newest — price is a last-write-wins value, so coalescing is lossless in effect. Notifications are never coalesced.

7.9 Event Schemas

The Kafka event schemas are part of the contract surface and are versioned alongside the REST API. `BidAccepted v1` is authored once and consumed identically by the producer and the fraud consumer (**FR-FRAUD-1**).

```
BidAccepted:
  type: object
  required: [event_id, schema_version, auction_id, bidder_id, amount,
             server_seq, accepted_at]
  properties:
    event_id:           { type: string, format: uuid }
    schema_version:     { type: integer, example: 1 }
    auction_id:         { type: string, format: uuid }   # partition key
    bidder_id:          { type: string, format: uuid }
    amount:             { type: string, example: "1250.00" }
    server_seq:         { type: integer, format: int64 }
    accepted_at:        { type: string, format: date-time }
    idempotency_key:    { type: string }
    device_fingerprint: { type: string, nullable: true }
```

Listing 7.7: `bid.events` — `BidAccepted v1` (**FR-BID-7**, **FR-FRAUD-1**)

```
SettlementStep:
  type: object
  required: [event_id, schema_version, order_id, type, saga_id]
  properties:
    event_id:       { type: string, format: uuid }
    schema_version: { type: integer, example: 1 }
    order_id:       { type: string, format: uuid }        # partition key
    type:
      type: string
      enum: [INVOICED, CAPTURED, HELD, RELEASED, REFUNDED, COMPENSATED]
    gross: { type: string }
    fee:   { type: string }
    net:   { type: string }
    saga_id:         { type: string, format: uuid }
    idempotency_key: { type: string }

ShillAlert:
  type: object
  required: [alert_id, schema_version, auction_id, score, raised_at]
  properties:
    alert_id:       { type: string, format: uuid }
    schema_version: { type: integer, example: 1 }
    auction_id:     { type: string, format: uuid }        # partition key
    ring_id:        { type: string, format: uuid, nullable: true }
    score:          { type: number, minimum: 0, maximum: 1 }
    signals:        { type: object }
    raised_at:
      type: string
      format: date-time
      description: Must precede the auction close time (FR-FRAUD-4, AC-14).
```

Listing 7.8: `payment.events` and `fraud.alerts` event schemas

7.10 Contract Testing

Three boundaries are covered by consumer-driven contract tests that fail CI when either side of a published shape changes (**FR-TEST-1**, **NFR-MAINT-3**):

Table 7.3: Contract-tested boundaries

ID	Boundary	What the test pins
CT-1	Bid Processing → Auction Orchestration	The state response shape and the set of states that permit a bid.
CT-2	Auction Orchestration → KYC listing gate	The verification status vocabulary and the refusal code returned to an unverified seller.
CT-3	Bid Processing → Pricing Engine	The increment response shape and the numeric type used for money.

Chapter 8

UI/UX Design — Final Mockups

The wireframes in the SRS were an initial low-fidelity draft. The mockups in this chapter are the final design: they specify layout, state, copy and interaction for every user-facing surface, and they are the reference the frontend implementation builds against (**FR-UI-1**–**FR-UI-6**).

8.1 Design Tokens and the Component Library

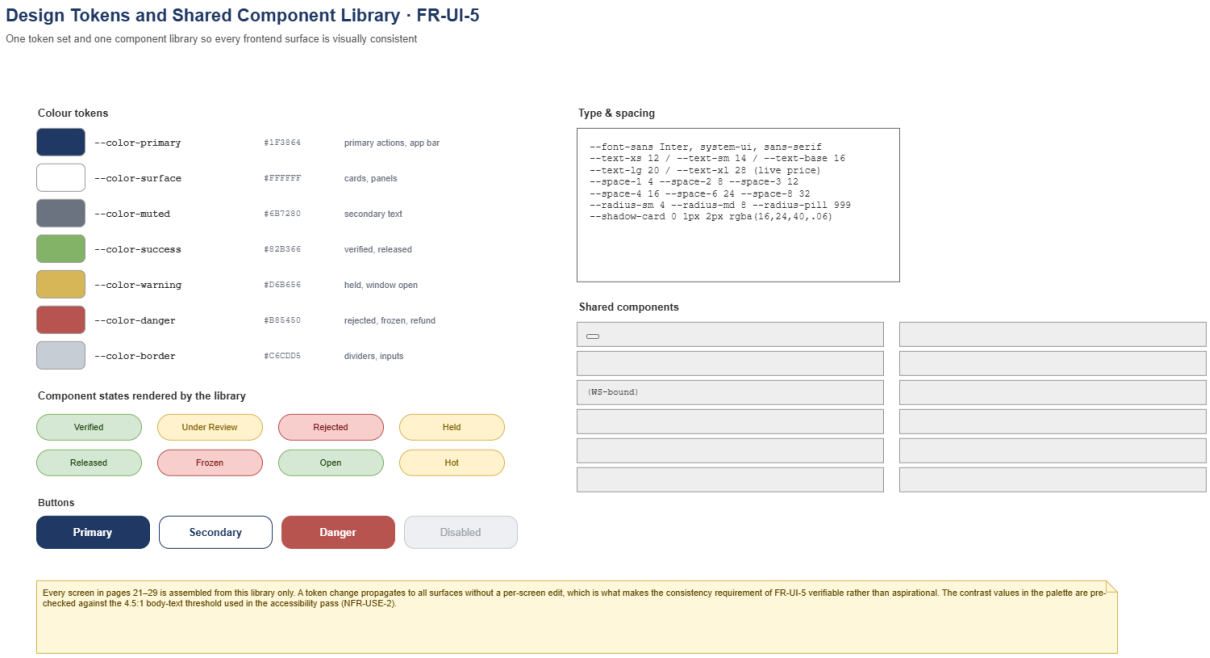


Figure 8.1: Design tokens and the shared component library

Every screen that follows is assembled from one component library over one token set (**FR-UI-5**). This is what makes visual consistency verifiable rather than aspirational: a token change propagates to all surfaces without a per-screen edit, and a reviewer can check consistency by checking the library rather than by inspecting nine screens.

The palette values are pre-checked against the 4.5:1 body-text contrast threshold used in the accessibility pass, so a screen assembled from the library starts compliant rather than being remediated afterwards.

8.2 Auction Browse and Search

Wireframe — Auction Browse & Search (Buyer) · FR-SEARCH-1, FR-SEARCH-2, FR-UI-5, FR-UI-6

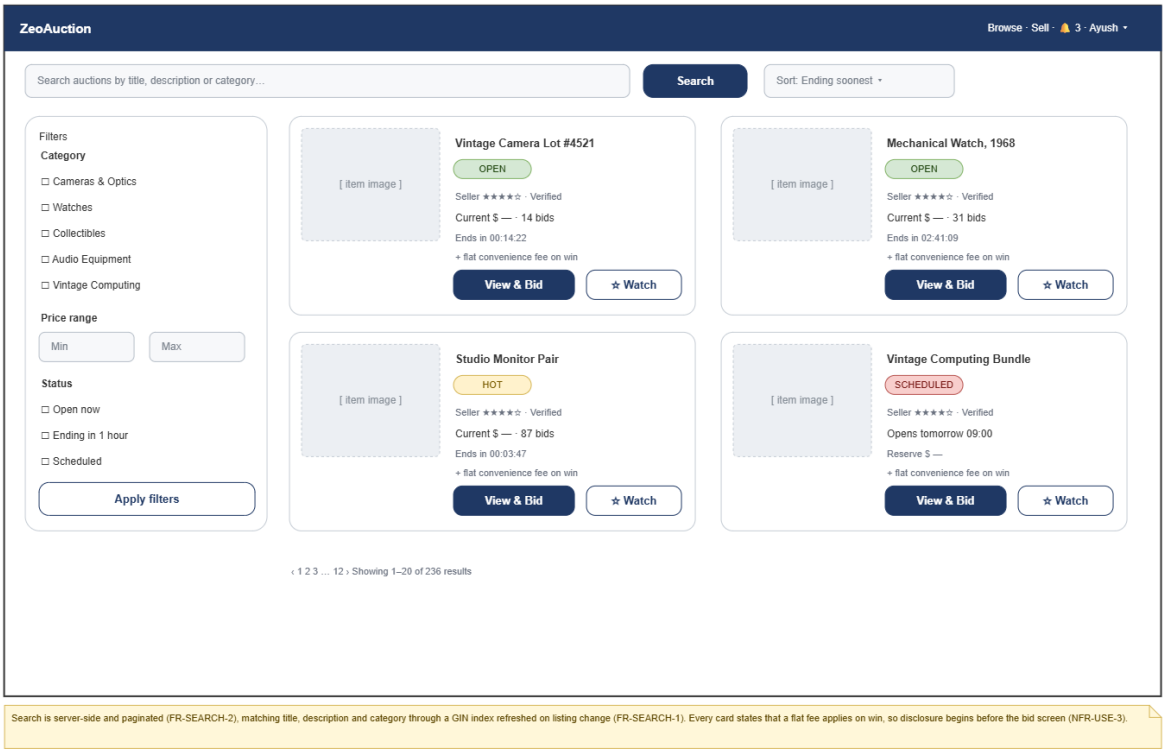


Figure 8.2: Final mockup: auction browse and search with faceted filters

Search is server-side and paginated (**FR-SEARCH-2**), matching title, description and category through a GIN index refreshed incrementally on listing change (**FR-SEARCH-1**). Each card states that a flat fee applies on win, so fee disclosure begins before the buyer ever reaches the bid screen (**NFR-USE-3**).

8.3 Auction Detail and Live Bidding

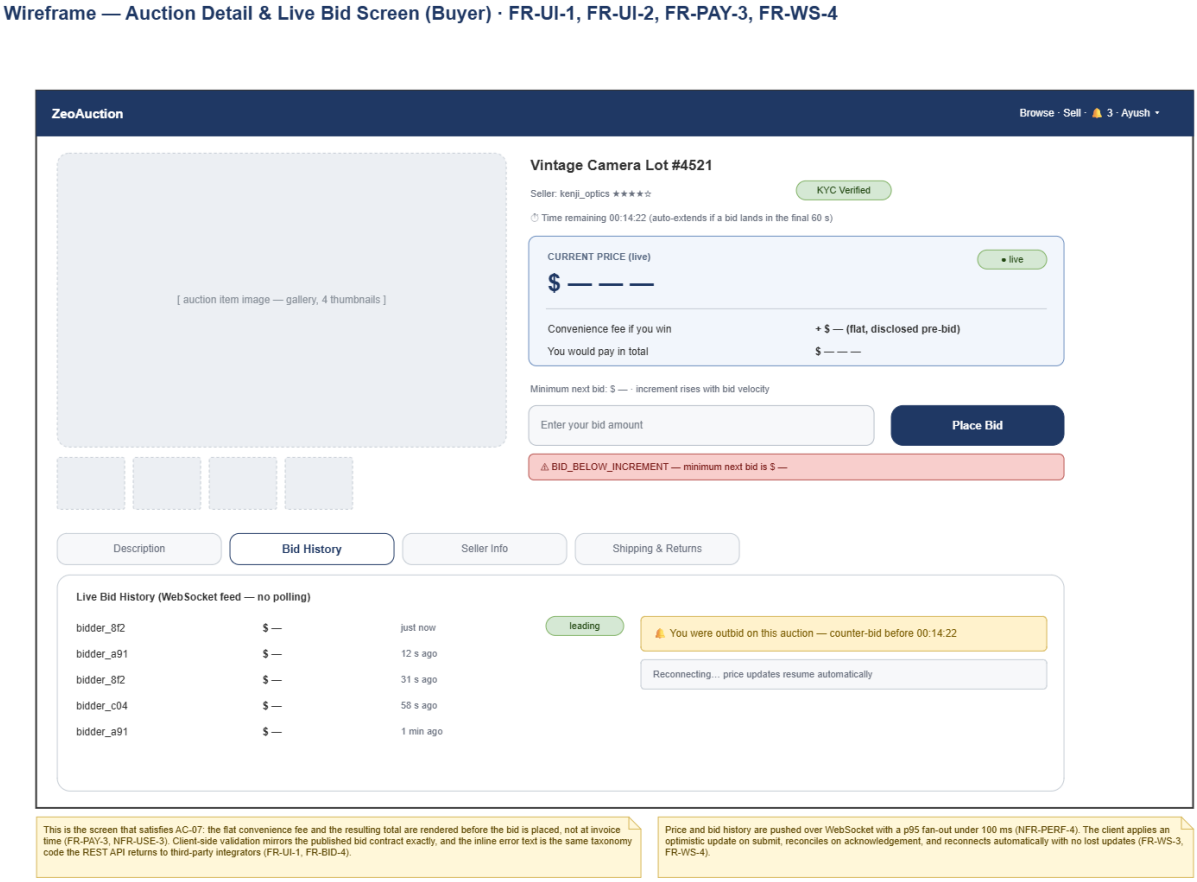


Figure 8.3: Final mockup: auction detail and live bid screen with fee disclosure and WebSocket bid history

This is the screen **AC-07** is verified against. The flat convenience fee and the resulting total are rendered before the bid is placed, not at invoice time (**FR-PAY-3**, **FR-UI-2**).

Three design decisions carry requirements:

1. **Client validation mirrors the published contract.** The inline error text is generated from the same rejection taxonomy the REST API returns to third-party integrators (**FR-UI-1**, **FR-BID-4**). The client never invents a rule the server does not enforce, and never enforces a rule the server does not.
2. **Price and history are pushed, not polled.** The bid history is a live WebSocket feed with a p95 fan-out target under 100 ms (**NFR-PERF-4**). The client applies an optimistic update on submit and reconciles on acknowledgement, so the interface feels immediate without lying about the outcome.
3. **Disconnection is a visible, recoverable state.** A reconnect banner appears rather than the price silently freezing, and updates resume from the last seen sequence number with no gap (**FR-WS-3**, **FR-WS-4**).

8.4 Seller KYC Submission

Wireframe — Seller KYC Submission Flow · FR-KYC-1, FR-KYC-6, NFR-SEC-2

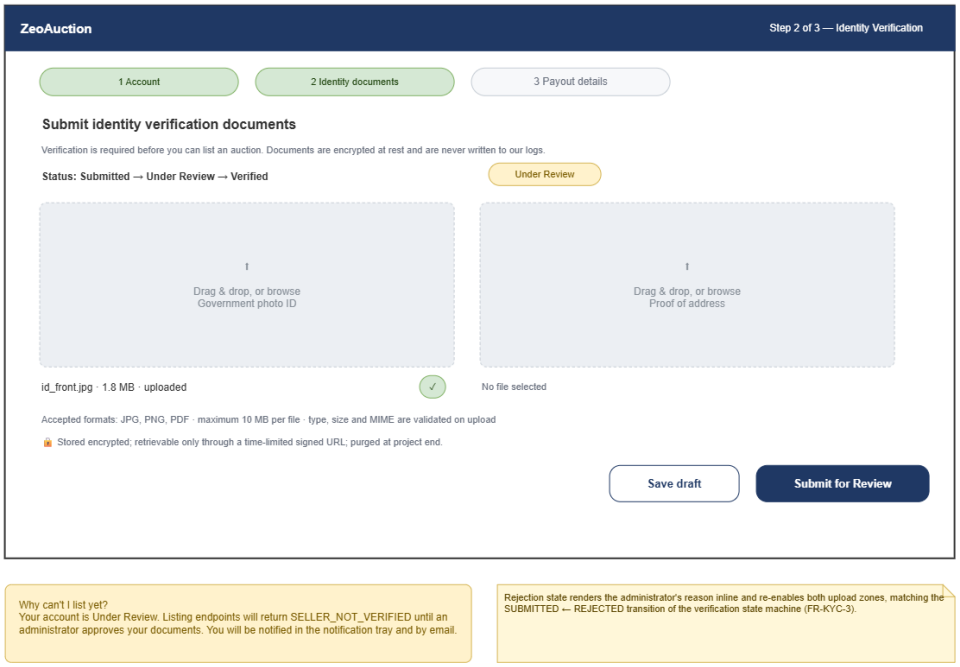


Figure 8.4: Final mockup: seller KYC document submission with verification-state feedback

The status strip reflects the verification state machine directly (**FR-KYC-3**, **FR-KYC-6**), so the seller sees the same states the system records. The screen states plainly that documents are encrypted at rest, reachable only through a time-limited signed URL, and never written to logs (**NFR-SEC-2**).

The rejected state renders the administrator's reason inline and re-enables both upload zones, matching the `REJECTED` → `SUBMITTED` transition. The "why can't I list yet" panel names the exact API refusal a seller would hit, so the UI explanation and the service behaviour agree.

8.5 Seller Dashboard

Wireframe — Seller Dashboard · FR-SEARCH-3, US-06

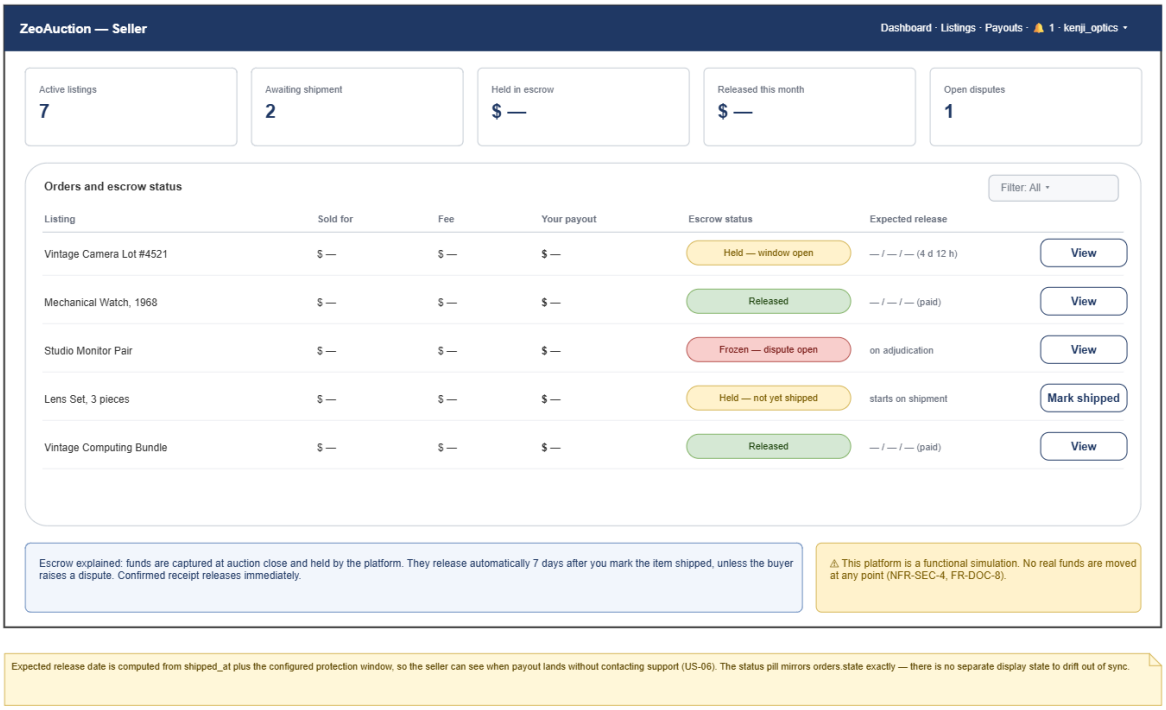


Figure 8.5: Final mockup: seller dashboard showing listings, sales, escrow state and expected payout date

The dashboard answers the question sellers actually have — when will I be paid — by computing the expected release date from `shipped_at` plus the configured protection window (**FR-SEARCH-3**, **US-06**). The status pill mirrors `orders.state` exactly; there is no separate display state that could drift out of sync with the ledger.

The simulation notice is displayed here rather than buried in documentation, because this is the screen where a user would otherwise form a belief about real money (**NFR-SEC-4**, **FR-DOC-8**).

8.6 Buyer Dispute Submission

Wireframe — Buyer Dispute Submission (window-gated) · FR-UI-3, FR-DISP-2

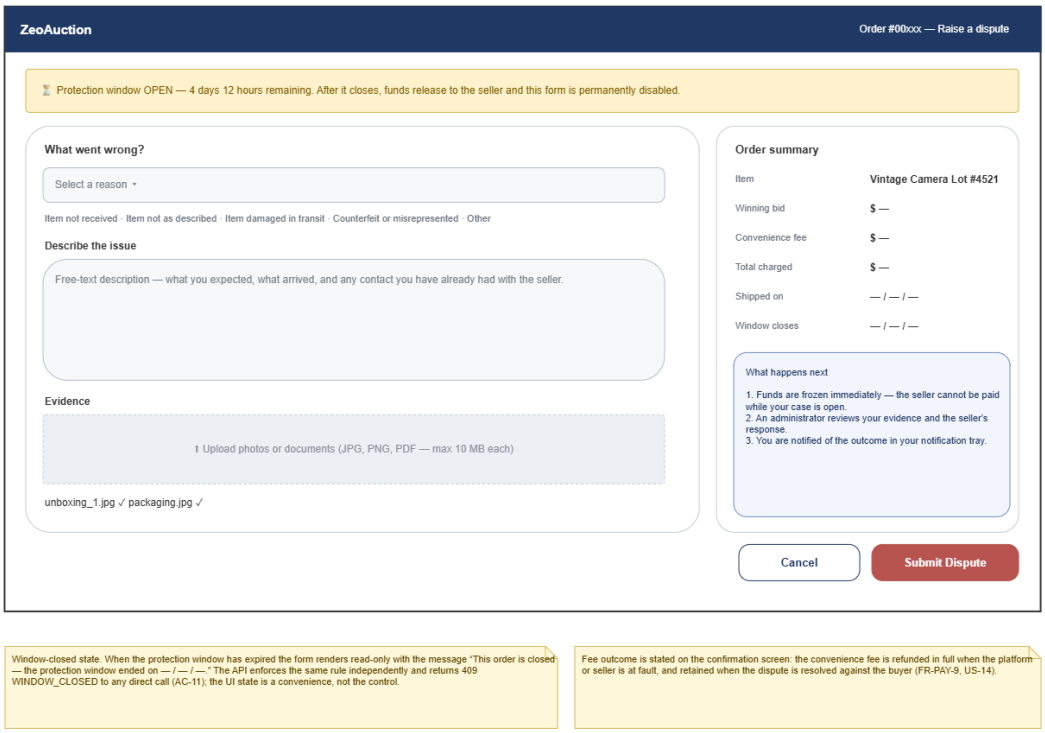


Figure 8.6: Final mockup: window-gated dispute submission with evidence upload

The form is enabled only while the protection window is open (**FR-UI-3**). When the window has expired the form renders read-only with the closing date. The UI state is a convenience only: the API enforces the same rule independently and returns `WINDOW_CLOSED` to any direct call (**AC-11**).

The order summary panel restates the fee, and the confirmation screen states the fee outcome, so a buyer knows before submitting that the fee is refunded in full on platform or seller fault and retained if the dispute is resolved against them (**FR-PAY-9**, **US-14**).

8.7 Administrator Dispute Adjudication

Wireframe — Administrator Dispute Adjudication Console · FR-DISP-4, FR-ADMIN-3

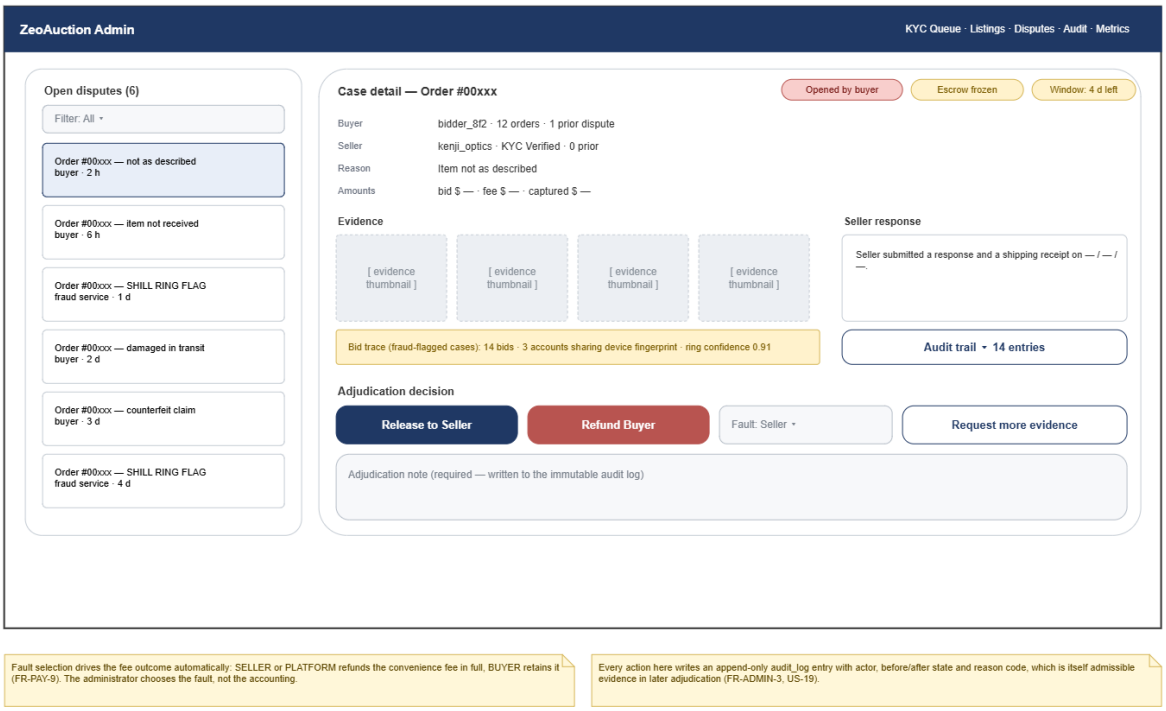


Figure 8.7: Final mockup: administrator dispute adjudication console

The console presents the case, the buyer's evidence, the seller's response and the audit history in one view, so an adjudication decision is made from evidence rather than from memory (**FR-DISP-4**, **US-18**). Fraud-flagged cases additionally render the linked bid trace — accounts, shared fingerprints and ring confidence — so the administrator does not have to reconstruct the pattern the detector already found (**FR-DISP-6**).

Fault selection and outcome are separate controls, because they answer different questions. The outcome decides who gets the money; the fault decides the fee treatment, and the system derives that automatically (**FR-PAY-9**). Every action writes an append-only audit entry with actor, before and after state, and reason code (**FR-ADMIN-3**, **US-19**).

8.8 Administrator KYC Review and Moderation

Wireframe — Administrator KYC Review Queue & Moderation · FR-KYC-4, FR-ADMIN-1, FR-ADMIN-2

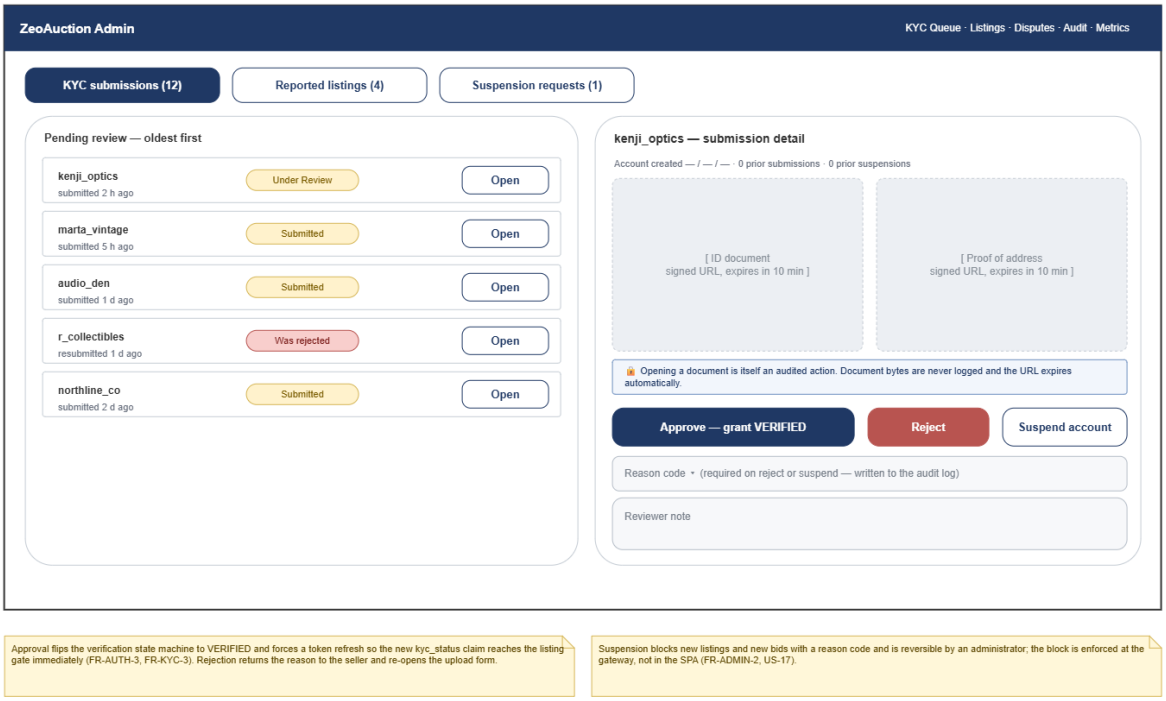


Figure 8.8: Final mockup: administrator KYC review queue, document viewer and moderation actions

Approval flips the verification state machine to `VERIFIED` and forces a token refresh, so the new `kyc_status` claim reaches the listing gate immediately (**FR-KYC-4**, **FR-AUTH-3**). Rejection and suspension both require a reason code, which is written to the audit log and returned to the user (**FR-ADMIN-2**).

Opening a KYC document is itself an audited action, and the signed URL expires automatically. The administrator's convenience does not extend the lifetime of the document's exposure.

8.9 Notification Tray, Login and Registration

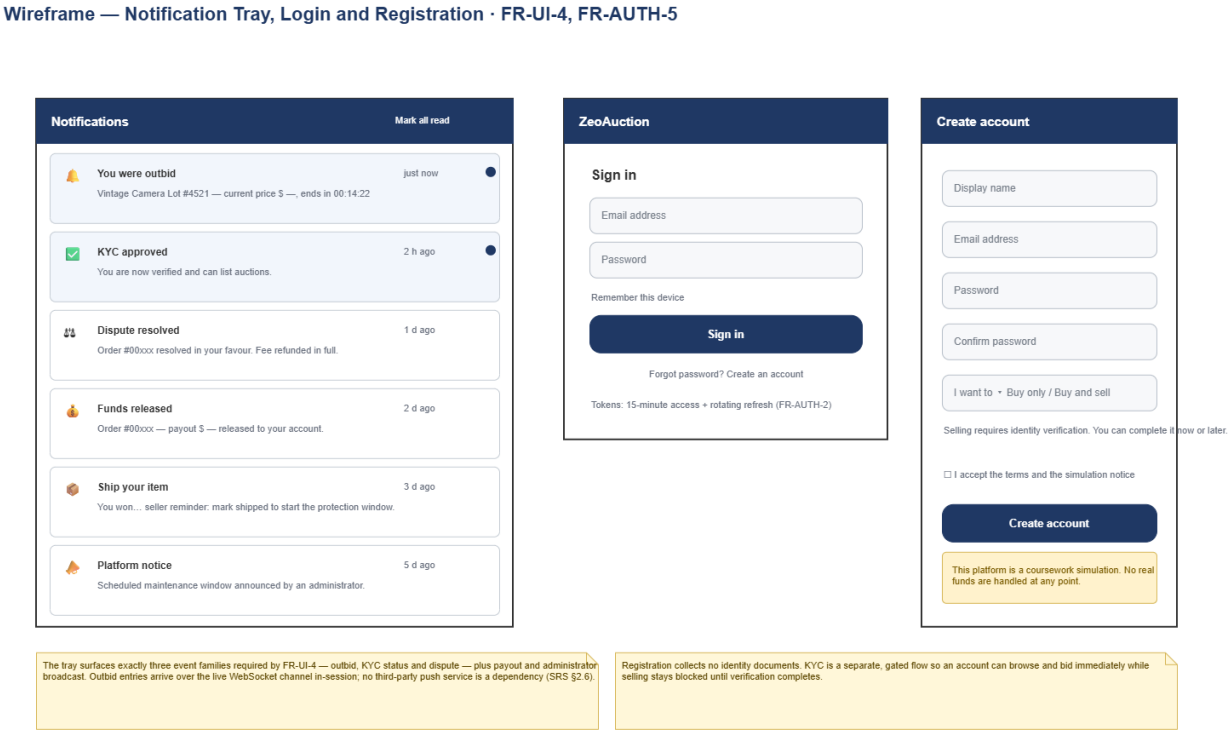


Figure 8.9: Final mockup: notification tray, sign-in and registration

The tray surfaces the three event families **FR-UI-4** requires — outbid, KYC status and dispute — plus payout and administrator broadcast. Outbid entries arrive over the platform's own WebSocket channel in-session; no third-party push service is a dependency.

Registration collects no identity documents. Verification is a separate gated flow, so a new account can browse and bid immediately while selling stays blocked until verification completes. The simulation notice appears at the point of account creation.

8.10 Responsive Layout and Accessibility

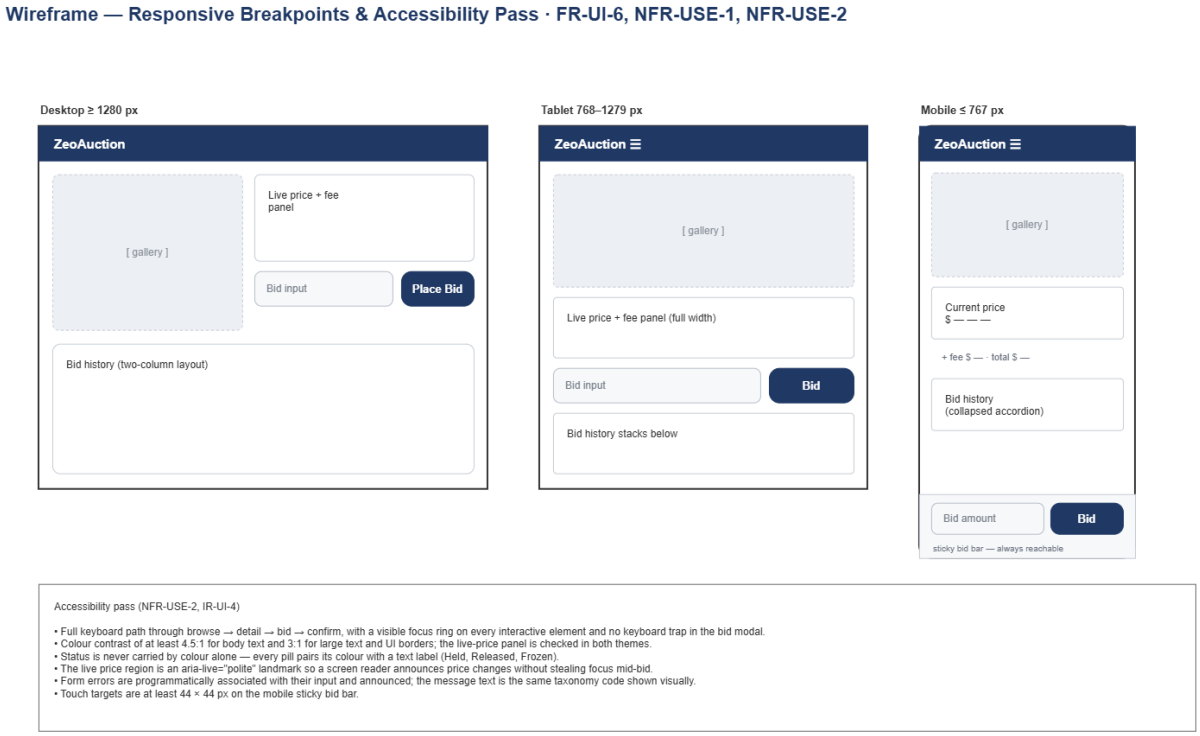


Figure 8.10: Final mockup: responsive breakpoints and the accessibility pass

Three breakpoints are specified (**FR-UI-6**, **NFR-USE-1**). The mobile layout keeps the bid control in a sticky bar rather than below the fold, because a bid placed too late is functionally a bid not placed.

The accessibility pass (**NFR-USE-2**, **IR-UI-4**) covers:

- a complete keyboard path through browse, detail, bid and confirm, with a visible focus ring and no keyboard trap in the bid modal;
- contrast of at least 4.5:1 for body text and 3:1 for large text and UI borders;
- status never carried by colour alone — every pill pairs colour with a text label;
- the live price region marked `aria-live="polite"` so a screen reader announces changes without stealing focus mid-bid;
- form errors programmatically associated with their input, using the same taxonomy message shown visually;
- touch targets of at least 44×44 px on the mobile bid bar.

8.11 Screen-to-Requirement Map

Table 8.1: User-facing screens and the requirements each realises

Screen	Figure	Requirements
Browse and search	8.2	FR-SEARCH-1, FR-SEARCH-2, FR-UI-5, NFR-USE-3
Auction detail and bid	8.3	FR-UI-1, FR-UI-2, FR-PAY-3, FR-WS-4, AC-07
Seller KYC upload	8.4	FR-KYC-1, FR-KYC-6, NFR-SEC-2
Seller dashboard	8.5	FR-SEARCH-3, US-06, NFR-SEC-4
Dispute submission	8.6	FR-UI-3, FR-DISP-2, FR-PAY-9, AC-11
Admin adjudication	8.7	FR-DISP-4, FR-DISP-6, FR-ADMIN-3
Admin KYC and moderation	8.8	FR-KYC-4, FR-ADMIN-1, FR-ADMIN-2
Notification tray and auth	8.9	FR-UI-4, FR-AUTH-5
Responsive and accessibility	8.10	FR-UI-6, NFR-USE-1, NFR-USE-2
Tokens and components	8.1	FR-UI-5

Chapter 9

Cross-Cutting Design Concerns

9.1 Security and Privacy

Table 9.1: Security controls and where each is implemented

Control	Implementation	Requirement
Authentication	JWT access tokens (15 min) plus rotating refresh tokens; rotation recorded so a replayed refresh token is detectable	NFR-SEC-1
Authorisation	Single gateway middleware evaluating role, `kyc_status` and suspension; services do not re-implement it	FR-AUTH-4
Document protection	Server-side encryption at rest, signed URLs with short TTL, object keys never rendered as URLs, no document bytes in logs	NFR-SEC-2
Data minimisation	Only synthetic or team-consented KYC samples; retention and purge recorded in an ADR	NFR-SEC-3, FR-DOC-4
Payment safety	Stripe test mode only; escrow ledger is an internal simulated record, stated in the UI and in the user manual	NFR-SEC-4, FR-DOC-5
Replay safety	Idempotency keys on bid submission, capture, release and refund; unique constraints back them in the database	NFR-SEC-5, FR-PAY-7
Transport	HTTPS/TLS for REST; WSS with an authenticated handshake for WebSocket	IR-COM-1, IR-COM-2
Audit	Append-only `audit_log` with actor, before/after and reason code; no `UPDATE` or `DELETE` grant	FR-ADMIN-3

9.2 Observability

Prometheus scrapes `/metrics` from every service; Grafana dashboards cover bid throughput, latency distribution and escrow-sweep health; Jaeger carries distributed traces across service boundaries so a cross-service settlement path is observable end to end (**NFR-OBS-1**, **NFR-OBS-2**, **FR-INFRA-7**).

Table 9.2: Instrumentation that exists to prove a specific requirement

Metric	Why it exists	Proves
`bids_accepted_total`, `bids_rejected_total{code}`	Throughput and the shape of the rejection mix under load	AC-01
`bid_ack_latency_seconds` histogram	p95 acknowledgement latency	AC-03
`ws_fanout_latency_seconds` histogram	Bid acceptance to watcher receipt	AC-05
`escrow_sweep_lag_seconds`	Delay between window expiry and release; alerts above 300 s	AC-12
`ledger_reconciliation_failures_total`	Must remain zero; any non-zero value fails the metric outright	AC-08
`fraud_alerts_before_close_ratio`	Proportion of alerts raised in time to act	AC-14
`circuit_breaker_state{boundary}`	Which fallbacks are active during a chaos test	AC-13

9.3 Performance Budgets

Table 9.3: Latency budget allocation on the bid acknowledgement path (target p95 500 ms)

Segment	Budget	Note
Gateway: TLS, JWT verify, routing	40 ms	No database access.
Idempotency `SETNX`	10 ms	Single Redis round trip.
Validation, including cross-service calls	180 ms	Circuit-broken; falls back to cached values rather than waiting.
Sequence allocation and price CAS	20 ms	Two Redis operations.
Enqueue for batched write and Kafka publish	60 ms	Publish is acknowledged by the broker; the durable row is not on this path.
Response serialisation and return	40 ms	
Allocated	**350 ms**	Leaving 150 ms of headroom for tail variance.

All figures are measured on the dedicated 8 vCPU / 16 GB host (**NFR-PERF-5**). Numbers from a shared-CPU hosting tier are recorded separately and are not used for acceptance.

9.4 Testing Design

Testability is a design obligation on every service, not an activity that happens afterwards. Each rig below exercises a specific design element and is specified alongside it.

Table 9.4: Test rigs and the design elements they exercise

Rig	What it exercises	Requirement
Contract suite	The three published boundaries CT-1 to CT-3; fails CI on either side	FR-TEST-1
End-to-end suite	Full lifecycle on the Compose stack: gated listing, bidding, settlement, hold, release	FR-TEST-2
Concurrency harness	200 bids/s against one contended auction, asserting identical winners across replays	FR-TEST-3, AC-02
Load and soak rig	1000 bids/s aggregate with 500 watchers for 10 minutes on the dedicated host	FR-TEST-4, AC-01
Fraud metric harness	Detector scored against the seeded 500-auction dataset; reports recall at fixed precision	FR-TEST-5, AC-14
Reconciliation test	1000 simulated settlements; exact ledger reconciliation with zero tolerance	FR-TEST-6, AC-08
Browser end-to-end	Bid, KYC and dispute journeys; admin approval, suspension and both dispute outcomes	FR-TEST-7
Chaos tests	One documented test per service: stop it, confirm the remainder serves and it recovers unaided	NFR-REL-2, AC-13

Chapter 10

Traceability

10.1 Requirement to Design Element

Table 10.1: Traceability from SRS requirement groups to the design elements that realise them

Requirements	Design elements	Location
FR-AUTH-1–5	Auth service, gateway middleware, JWT claim design, auth screens	Fig. 2.2, 4.2, 8.9; Sec. 7.2
FR-KYC-1–6	Verification state machine, encrypted document store, listing gate, upload and review screens	Fig. 6.3, 6.7, 8.4, 8.8; Sec. 4.2
FR-AO-1–6	Auction aggregate, lifecycle state machine and transition table, settlement trigger	Fig. 6.1, 5.3, 6.5
FR-BID-1–7	Bid component design, sequence allocator, Redis buffer, rejection taxonomy, event contract	Fig. 5.1, 6.4, 2.4; Sec. 5.1, 7.3
FR-PRICE-1–3	Velocity windows table, pricing contract CT-3, hot-auction flag	Fig. 4.2, 2.3
FR-PAY-1–9	Payment port and adapters, saga steps, escrow ledger, fee engine, release triggers	Fig. 5.2, 4.3, 6.2, 6.5; Sec. 7.4
FR-DISP-1–7	Protection-window policy, freeze rule, adjudication path, sweep job, refund executor	Fig. 5.2, 6.2, 6.6, 8.6, 8.7
FR-FRAUD-1–6	Streaming consumer, signals, scorer, ring clusterer, dataset generator	Fig. 5.4, 2.4; Sec. 5.4

Requirements	Design elements	Location
FR-WS-1–4	WebSocket gateway component, message protocol, reconnect and backpressure design	Fig. 2.2, 6.4; Sec. 7.8
FR-SEARCH-1–3	Search vector and GIN index, search contract, dashboard projection	Fig. 4.2, 8.2, 8.5; Sec. 7.7
FR-ADMIN-1–4	Admin queues, suspension workflow, append-only audit log, metrics view	Fig. 4.3, 8.8, 8.7
FR-UI-1–6	Final mockups, token set and component library, responsive and accessibility specification	Ch. 8, Fig. 8.1–8.10
FR-INFRA-1–7	Compose stack, Render blueprint, Kafka private service, CI gates, cron job, observability	Fig. 3.1, 3.2; Ch. 3
FR-TEST-1–7	Contract, end-to-end, concurrency, load, fraud metric, reconciliation and browser rigs	Sec. 9.4
FR-DOC-1–8	Generated OpenAPI, ADR summary, settlement and retention decisions	Sec. 2.6, Ch. 7
NFR-PERF-1–5	Hot-path design, latency budget, dedicated measurement host	Sec. 5.1, 9.3, 3.3
NFR-SEC-1–5	Security control table, idempotency design, document handling	Sec. 9.1, 4.3
NFR-REL-1–4	Circuit-breaker fallbacks, failure behaviour table, saga compensations, ledger invariants	Sec. 2.4, 3.4, 4.3, 5.2
NFR-SCALE-1–2	Independent deployables, Redis buffering with batched persistence	Sec. 2.3, 4.4
NFR-MAINT-1–4	CI gates, review policy, contract tests, environment parity	Ch. 3, Sec. 7.9
NFR-USE-1–3	Responsive breakpoints, accessibility pass, pre-bid fee disclosure	Sec. 8.9, 8.2
NFR-OBS-1–2	Metric inventory and tracing	Sec. 9.2

Requirements	Design elements	Location
NFR-FRAUD-1	Dataset generator and metric harness built before the detector	Sec. 5.4

10.2 Design Coverage Check

Table 10.2: Design artefact inventory

Artefact	Count	Location
Architectural views (context, container, interaction, events)	4	Chapter 2
Deployment and pipeline views	2	Chapter 3
Entity–relationship models	2	Chapter 4
Data ownership map	1	Chapter 4
Component diagrams	2	Chapter 5
Class diagrams	2	Chapter 5
State machines	3	Chapter 6
Sequence diagrams	3	Chapter 6
Activity diagram	1	Chapter 6
OpenAPI contract fragments	8	Chapter 7
Endpoint inventory	44 endpoints	Chapter 7
Event schemas	4	Chapters 2, 7
Final UI mockups	9	Chapter 8
Design-token and component specification	1	Chapter 8
Total diagrams	**30**	Chapters 2–8